COSC 2633/2637
Big Data Processing

Week 2

Hadoop, HDFS, YARN

RMIT UNIVERSITY

Part I

Could Computing and Datacentre

No data like more data!

- Develop an improved grammar checker using four learners
  - Winnow
  - Perceptron
  - Naïve Bayes
  - Memory-based learner
- Researchers collected a 1-billion-word training corpus from a variety of English texts, including
  - news articles,
  - scientific abstracts,
  - government transcripts,
  - literature
  - other varied forms of prose.

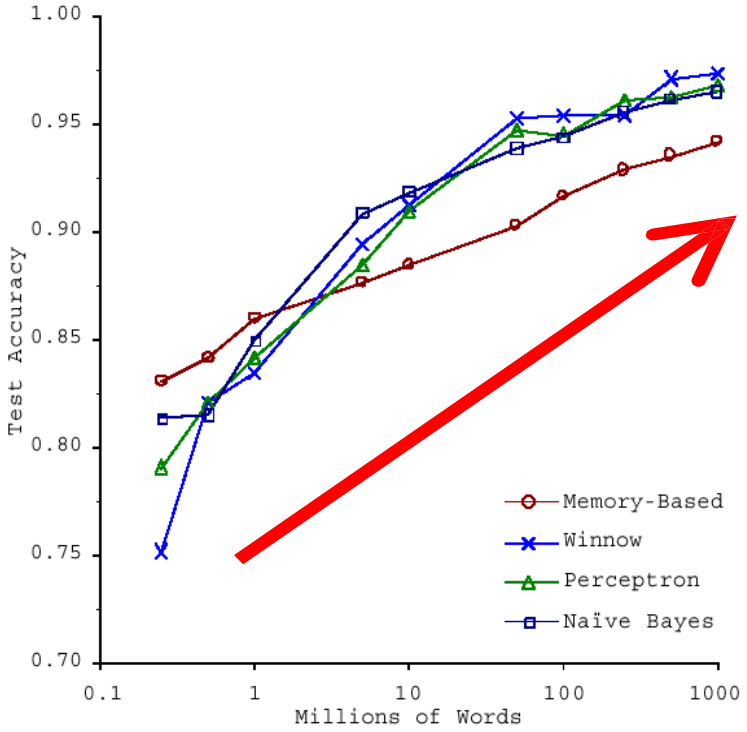


Figure shows learning curves for each learner, up to one billion words of training data.

(Banko and Brill, ACL 2001)



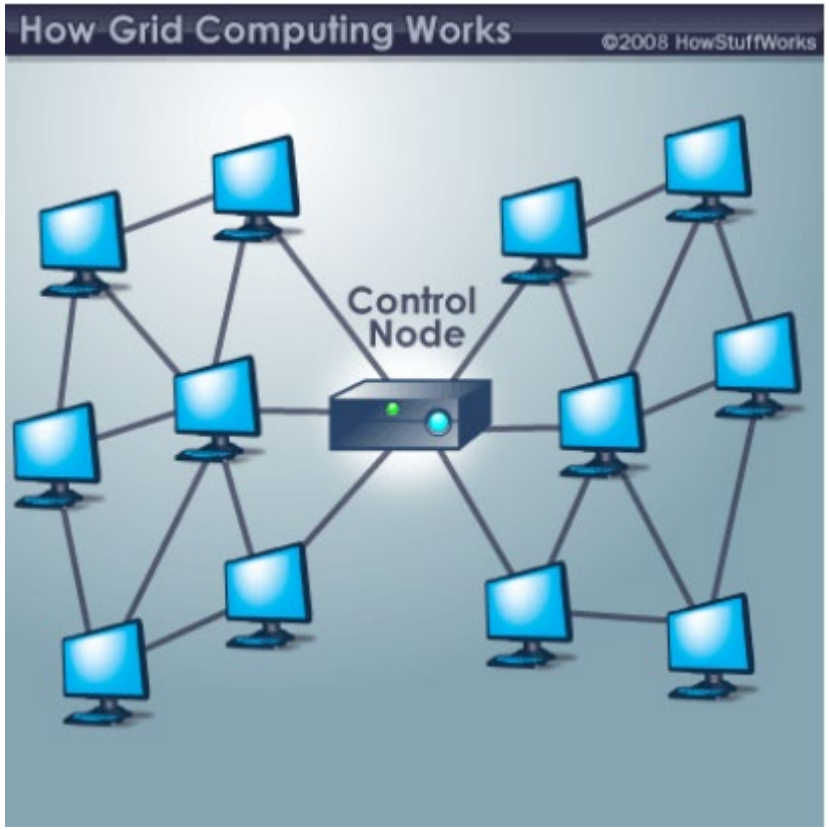
Interlude: Cloud Computing

Source: Wikipedia (Clouds)

The best thing since sliced bread?

Before clouds…

- **Grids**



In a basic grid computing system, every computer can access the resources of every other computer belonging to the network.
HOWSTUFFWORKS

The best thing since sliced bread?

Before clouds…

- **Grids**
- **Connection machine**

A Connection Machine (CM) is a member of a series of massively parallel supercomputers that grew out of doctoral research on alternatives to the traditional von Neumann architecture of computers by Danny Hillis at the Massachusetts Institute of Technology (MIT) in the early 1980s.



CM-1



CM-5

Starting with CM-1, the machines were intended originally for applications in artificial intelligence (AI) and symbolic processing, but later versions (CM-5) found greater success in the field of computational science.

The best thing since sliced bread?

Before clouds…

- **Grids**
- **Connection machine**
- **Vector supercomputers**

NEC SX Vector Supercomputer



NEC SX Aurora TSUBASA

In computing, a vector processor or array processor is a central processing unit (CPU) that implements an instruction set containing instructions that operate on one-dimensional arrays of data called vectors.

GPUs (Graphics processing unit) are Vector Processors

The best thing since sliced bread?

- Before clouds…
  - **Grids**
  - **Connection machine**
  - **Vector supercomputers**
  - **…**
- Cloud computing means many different things:
  - **Rebranding of web 2.0**
  - **Utility computing**
  - **Everything as a service**

Cloud Computing - Rebranding of Web 2.0

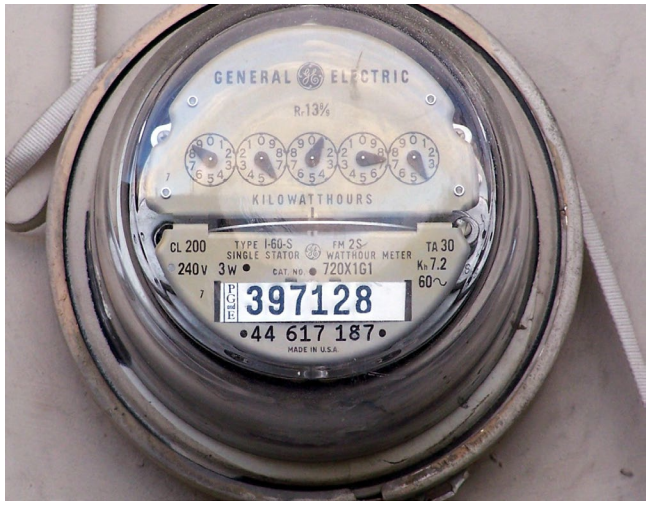
Web 2.0 encloses technologies and services that facilitate interactive sharing, collaboration, user-centered design, and application composition.

Clouds refer to the servers that run rich, interactive web 2.0 applications

examples: Facebook, Flicker, Linkedin, YouTube, Wikipedia, BlogSpot, Word press, Twitter, Pinterest, Google Docs, Instagram and many more.

Cloud Computing - Utility Computing

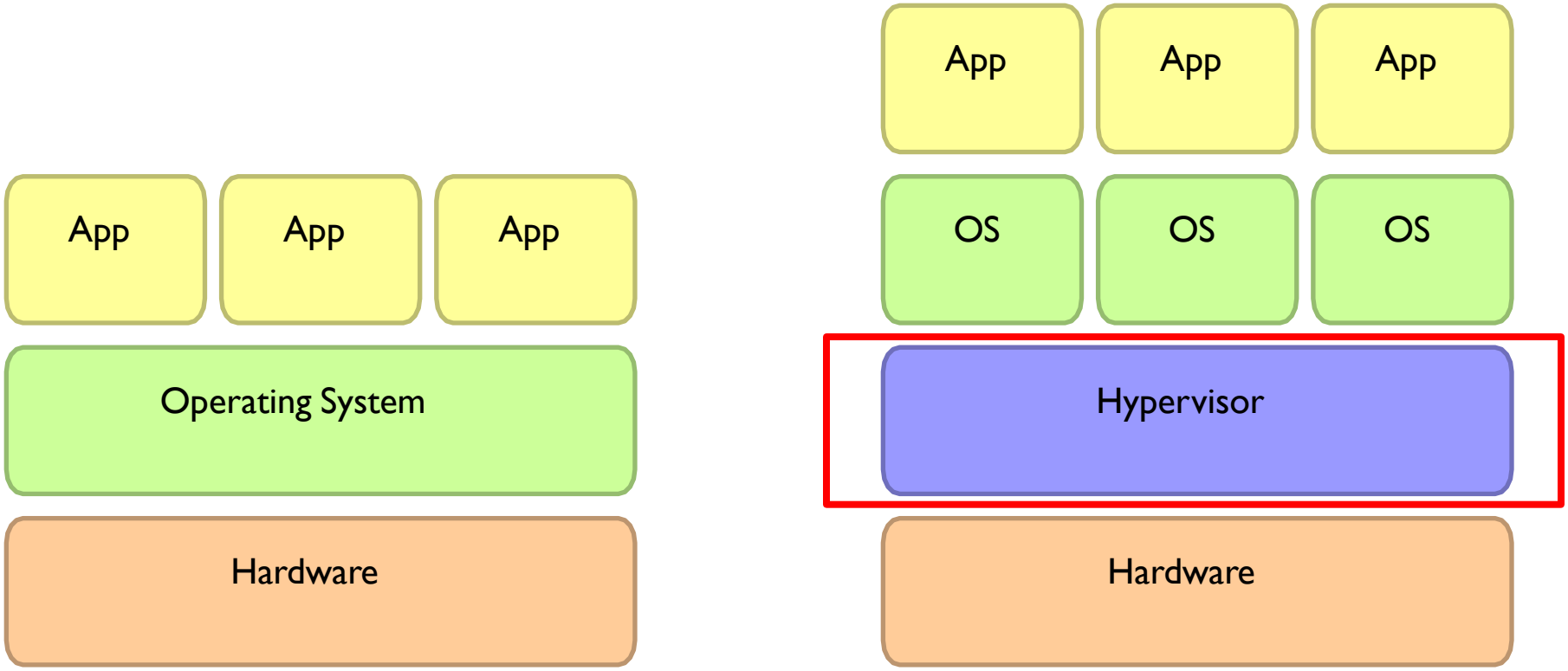
- What is Utility Computing?
  - Computing resources as a metered service ("pay as you go")
  - How to provide metered service?



The ability to dynamically provision virtual machines

Cloud Computing - Utility Computing

Enabling Technology: Virtualization



Cloud computing – Everything as a Service

Computing resources as a metered service

- Infrastructure as a Service (IaaS)

  Examples: DigitalOcean, Linode, Rackspace, Amazon Web Services (AWS), Cisco Metapod, Microsoft Azure, Google Compute Engine (GCE)

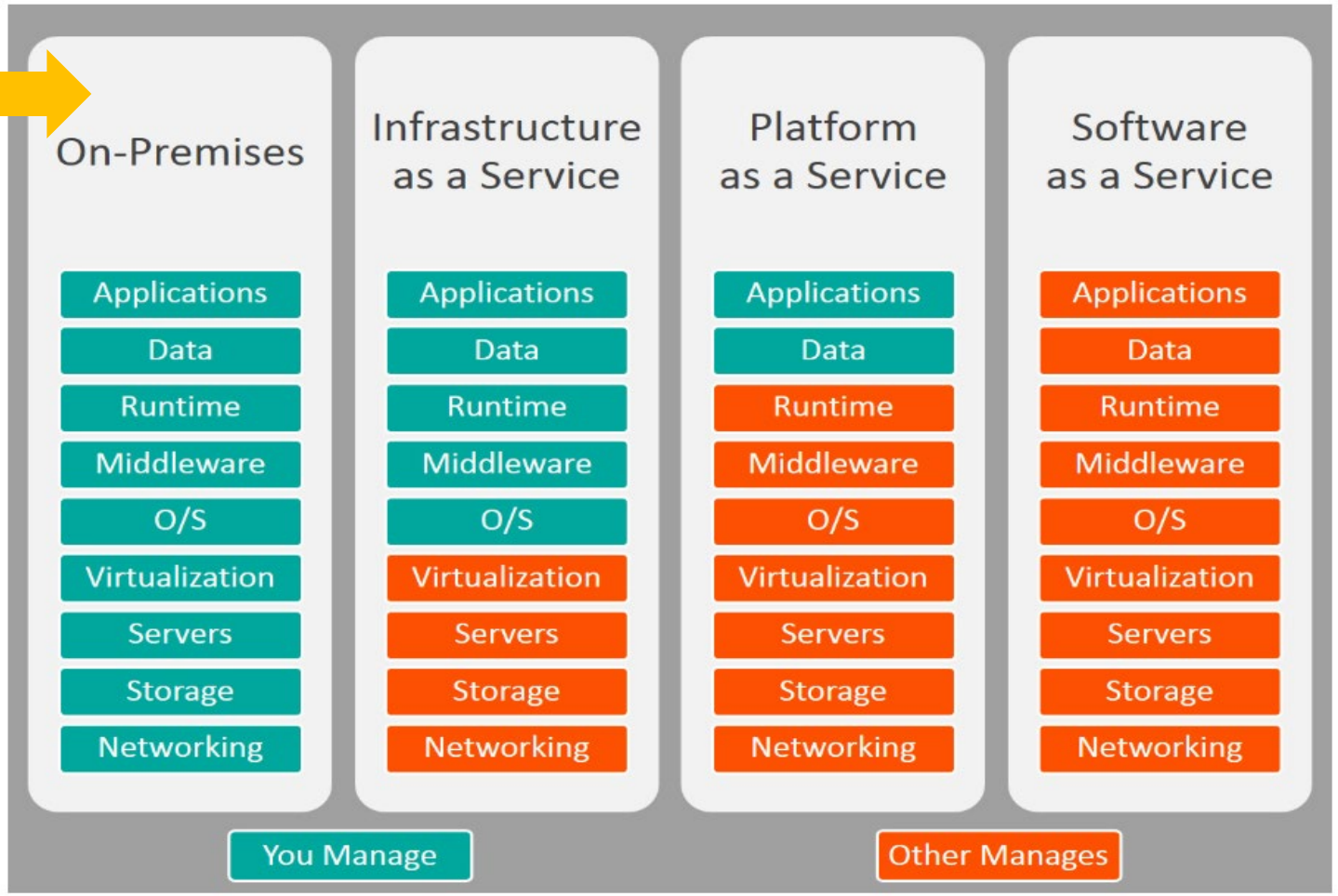
- Platform as a Service (PaaS)

  Example: AWS Elastic Beanstalk, Windows Azure, Heroku, Force.com, Google App Engine, Apache Stratos, OpenShift

- Software as a Service (SaaS)

  Example: Google Apps, Dropbox, Salesforce, Cisco WebEx, Concur, GoToMeeting

On-premises is applications installed and runs on computers on the premises of the person or organization using the software, rather than at a remote facility such as a server farm or cloud.

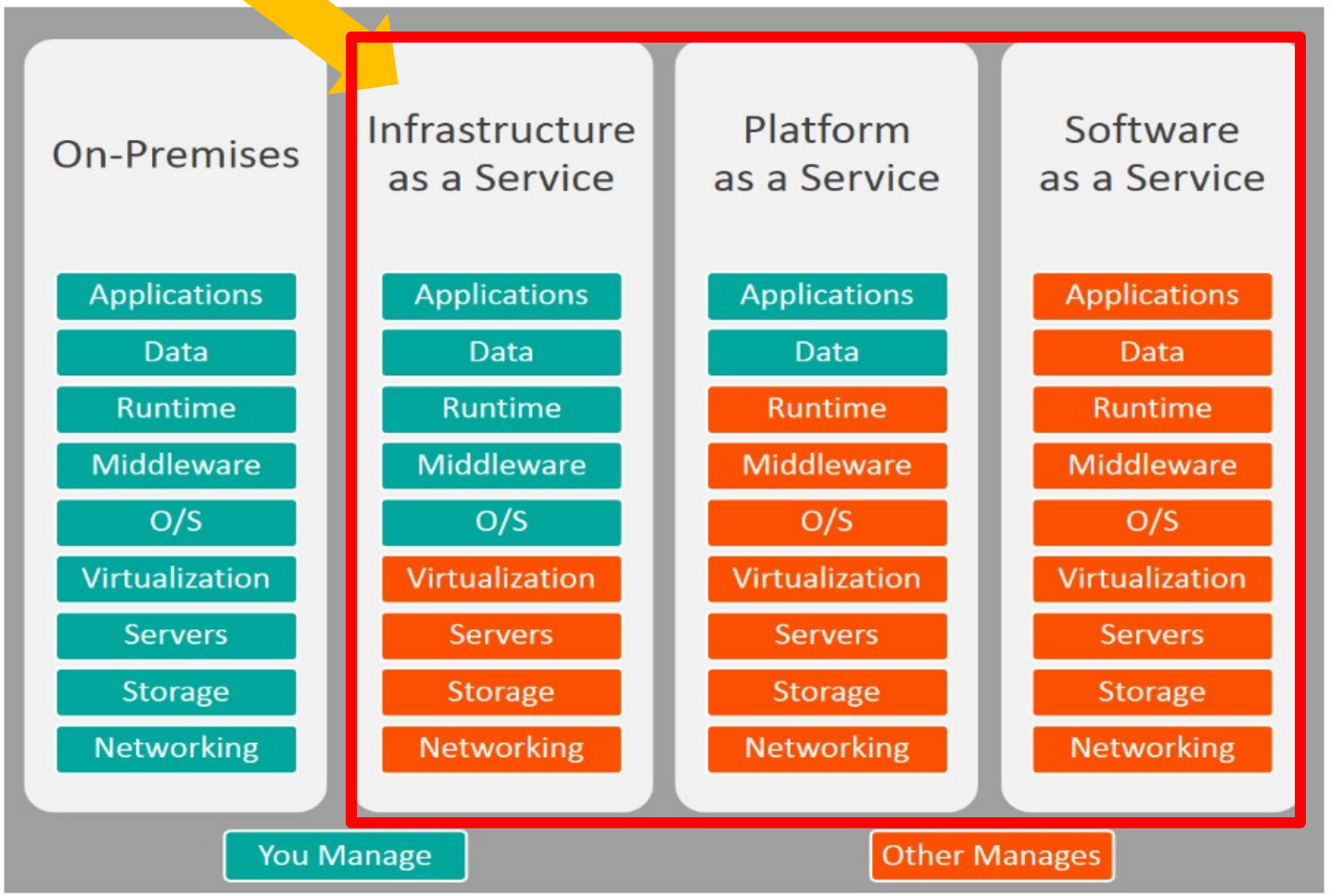


RMIT Classification: Trusted

If want to complete control over your entire infrastructure without having to physically maintain it

Infrastructure as a Service (IaaS)

Examples: DigitalOcean, Linode, Rackspace, Amazon Web Services (AWS), Cisco Metapod, Microsoft Azure, Google Compute Engine (GCE)

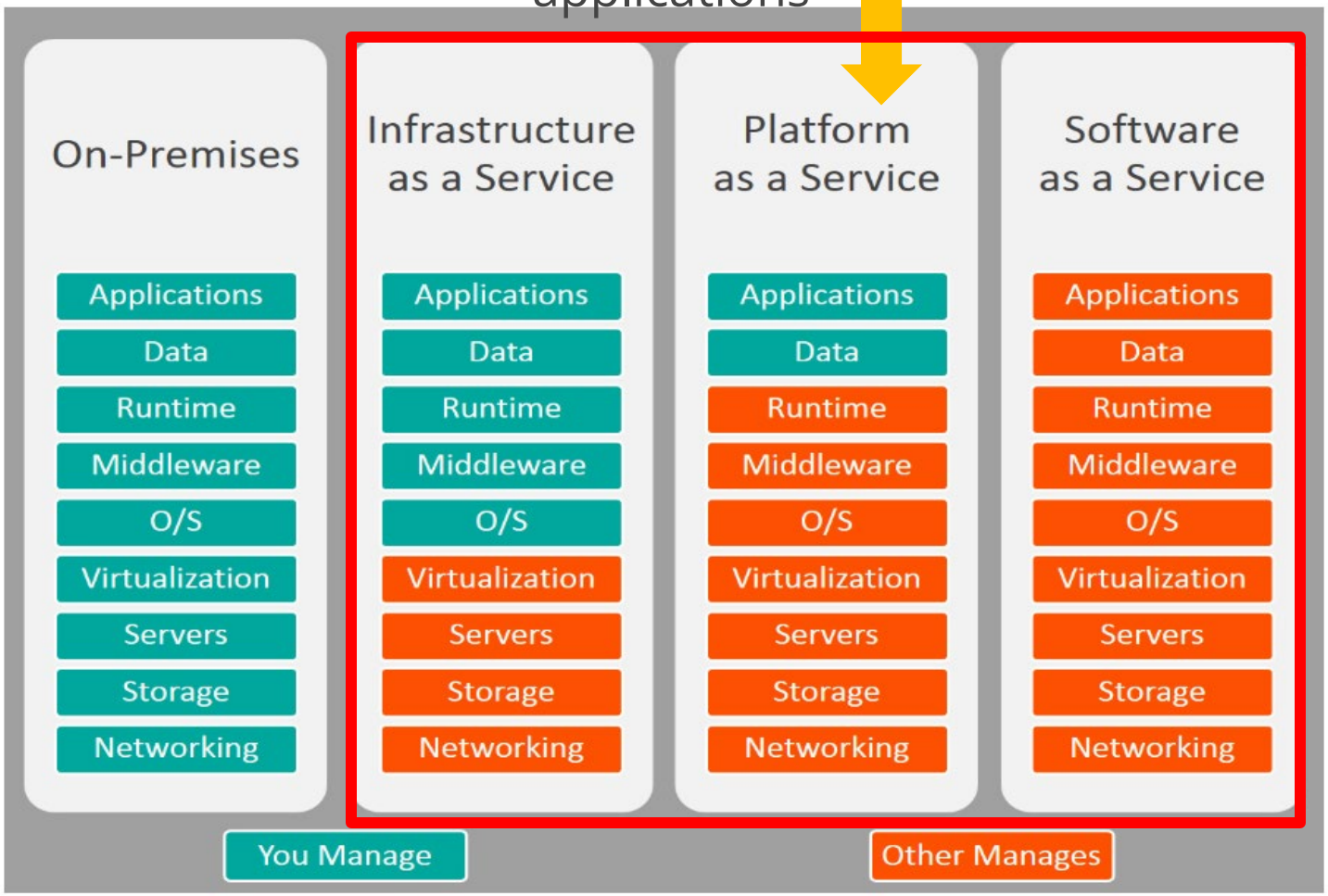


RMIT Classification: Trusted

Platform as a Service (PaaS)

Example: AWS Elastic Beanstalk, AWS EMR, Windows Azure, Heroku, Force.com, Google App Engine, Apache Stratos, OpenShift

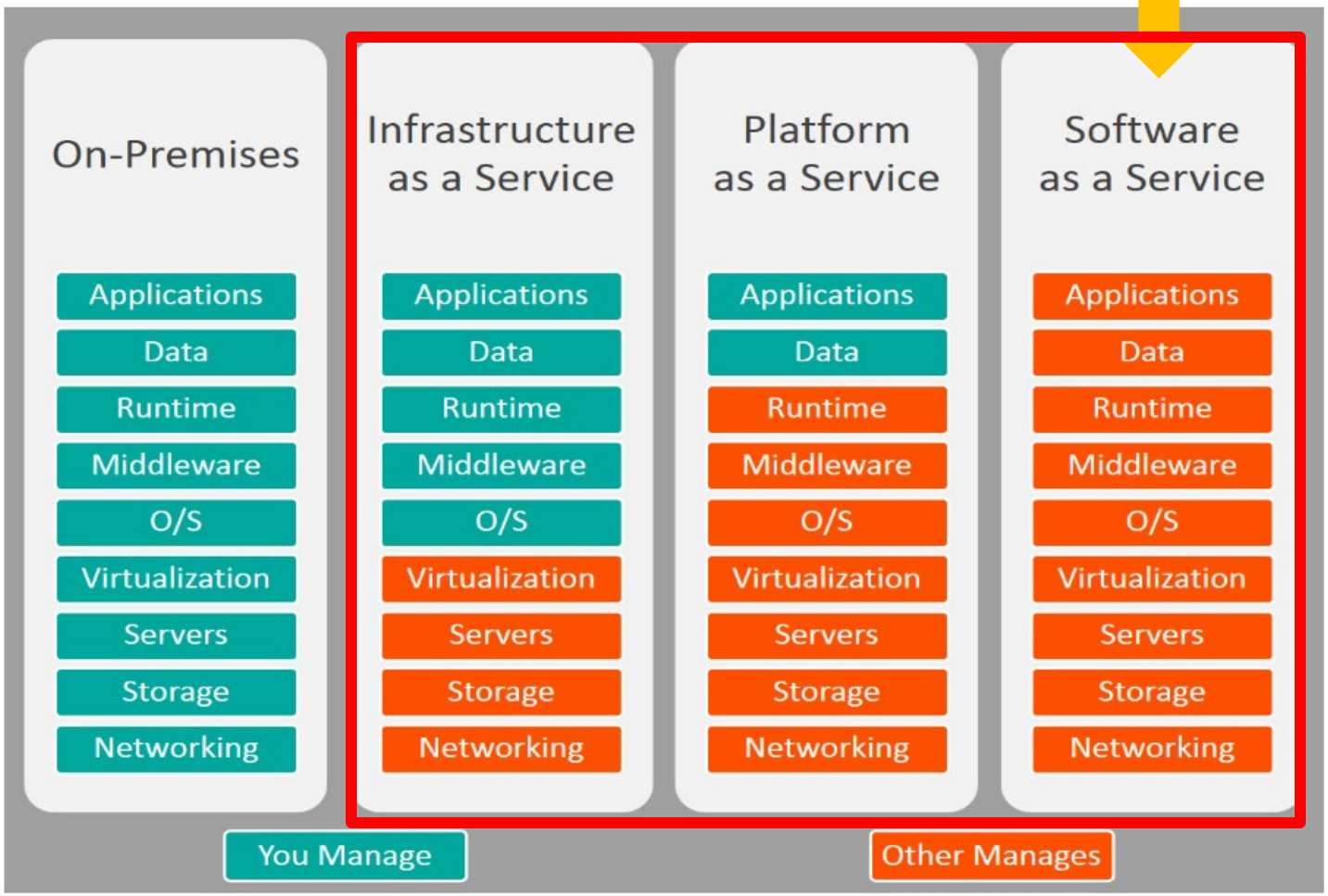
If you need a smooth platform that allows you to create customized applications



Software as a Service (SaaS)

Example: Google Apps, Dropbox, Salesforce, Cisco WebEx, Concur, GoToMeeting

If you are looking for cloud-based software for storage opt ns



The best thing since sliced bread?

- Before clouds…
  - **Grids**
  - **Connection machine**
  - **Vector supercomputers**
  - **…**
- Cloud computing means many different things:
  - **Rebranding of web 2.0**
  - **Utility computing**
  - **Everything as a service**

Cloud Computing
Ready for Big Data Processing

- Ready-made big data problems
  - Big data already in clouds
    - Social media, user-generated content
      Example: data of Facebook, Flicker, Linkedin, YouTube, Wikipedia, BlogSpot, Word press, Twitter, Pinterest, Google Docs, Instagram and many more.
  - Demand of business intelligence
    - Gather everything in a data warehouse and run analytics to generate insight
      Example: friend recommendation for users in Facebook, the most visited pages in Wikipedia, etc.
- Utility computing provides:
  - Ability to provision Hadoop clusters on-demand in the cloud
  - lower barrier to entry for tackling big data problems

Cloud Computing
Ready for Big Data Processing

- Ready-made big data problems
  - Big data already in clouds
    - Social media, user-generated content
      Example: data of Facebook, Flicker, Linked in, YouTube, Wikipedia, BlogSpot, Word press, Twitter, Pinterest, Google Docs, Instagram and many more.
  - Demand of business intelligence
    - Gather everything in a data warehouse and run analytics to generate insight
      Example: friend recommendation for users in Facebook, the most visited pages in Wikipedia, etc.
- Utility computing provides:
  - Ability to provision Hadoop clusters on-demand in the cloud
  - lower barrier to entry for tackling big data problems

Commoditization and democratization of big data capabilities

the action of making something can be bought and sold

Cloud Computing
Ready for Big Data Processing

- Ready-made big data problems
  - Big data already in clouds
    - Social media, user-generated content
      Example: data of Facebook, Flicker, Linked in, YouTube, Wikipedia, BlogSpot, Word press, Twitter, Pinterest, Google Docs, Instagram and many more.
  - Demand of business intelligence
    - Gather everything in a data warehouse and run analytics to generate insight
      Example: friend recommendation for users in Facebook, the most visited pages in Wikipedia, etc.
- Utility computing provides:
  - Ability to provision Hadoop clusters on-demand in the cloud
  - lower barrier to entry for tackling big data problems

Commoditization and democratization of big data capabilities

the action of making something accessible to everyone.

Cloud Data Centre

A cloud data centre is where the data are stored in cloud.

- Security
  - When your data is stored on cloud servers, it automatically gets fragmented and duplicated across various locations for secure storage.
    - In case there are any failures, your cloud services provider will make sure that there is a backup of your data.
- Cost – PAYG
- Accessibility – Internet
- Scalability – increase or decrease easily

Datacenter

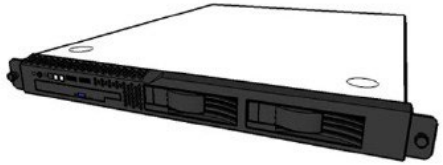


Source: Google

RMIT Classification: Trusted

Building Blocks

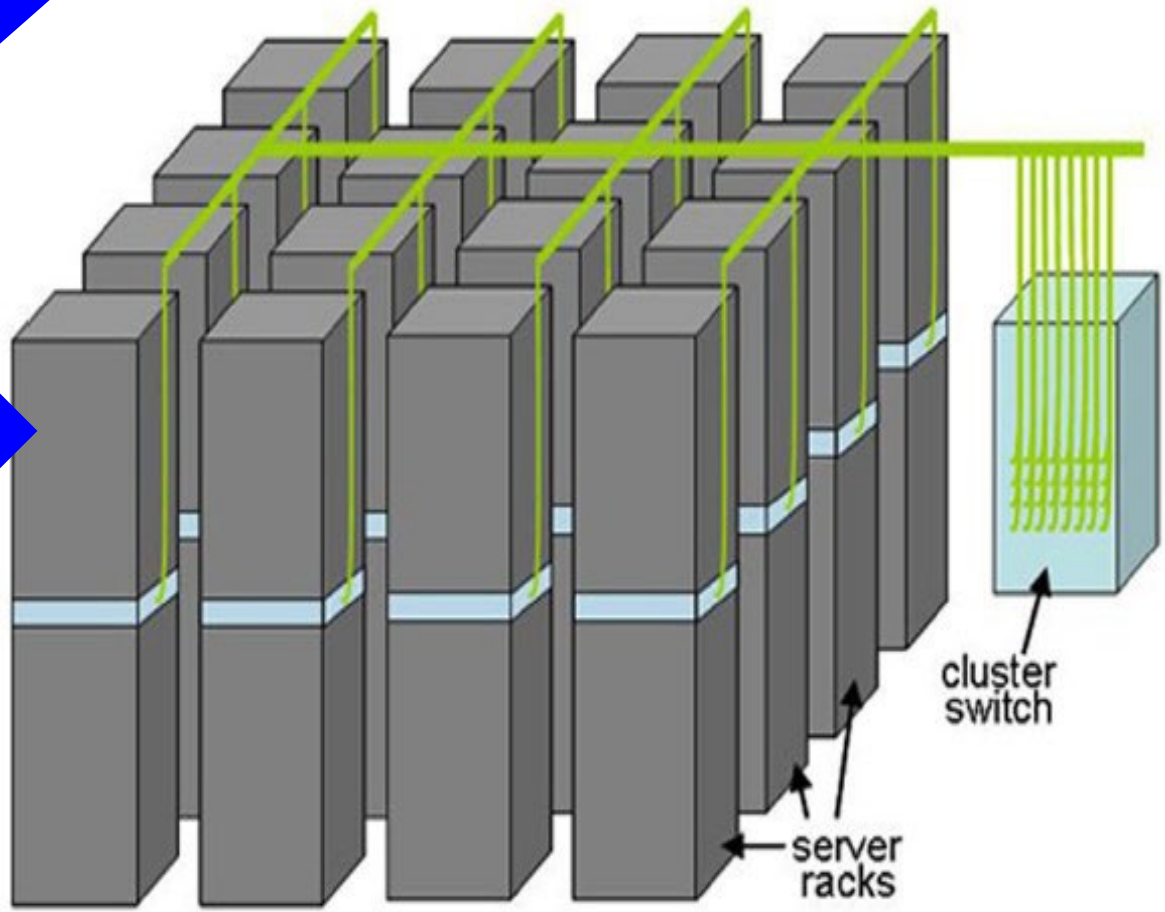




blade server

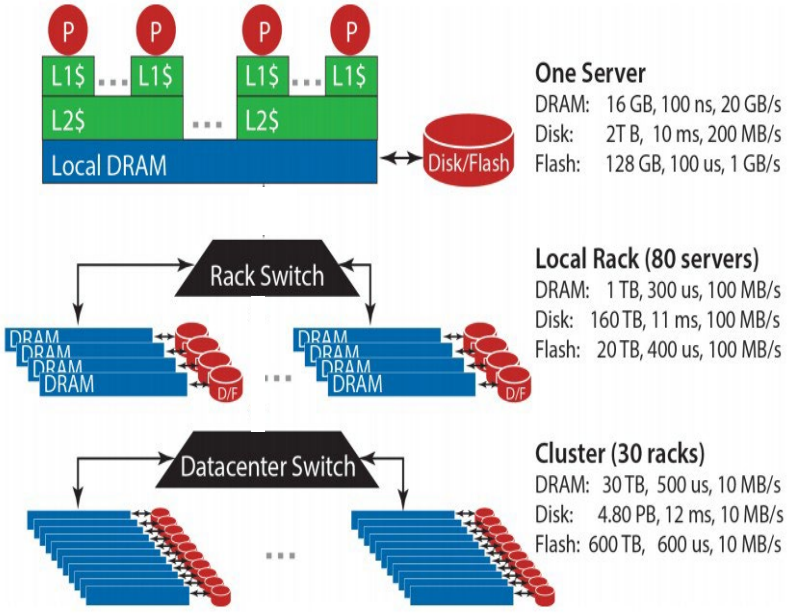
Rack

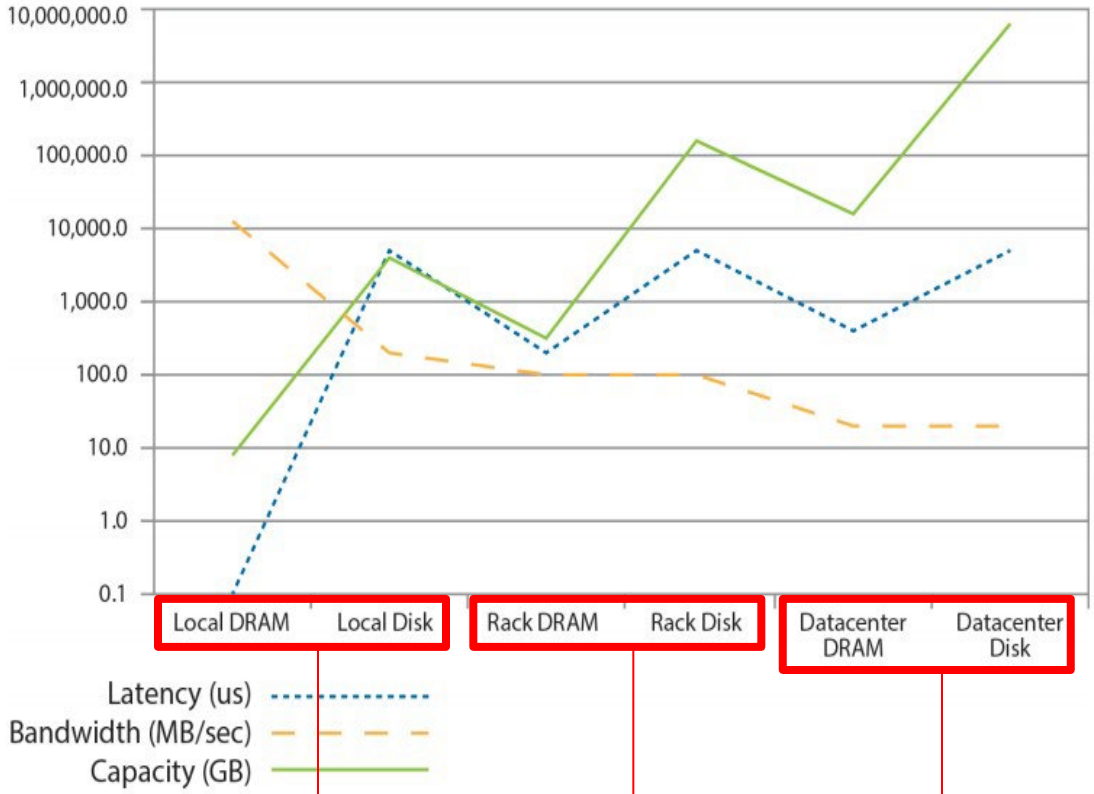




Source: Barroso and Urs Hölzle (2009)

Storage Hierarchy

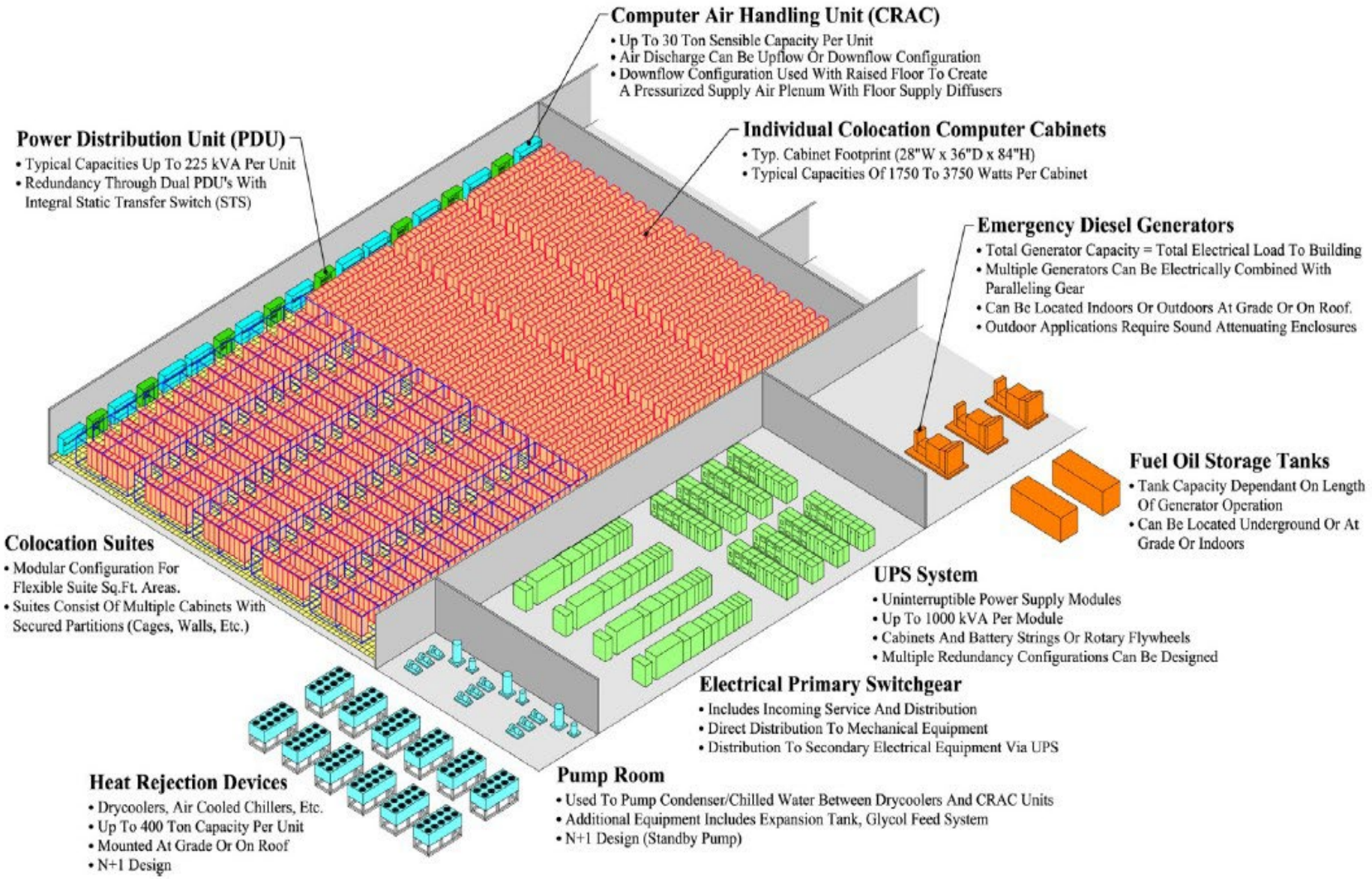




Source: Barroso and Urs Hölzle (2013)

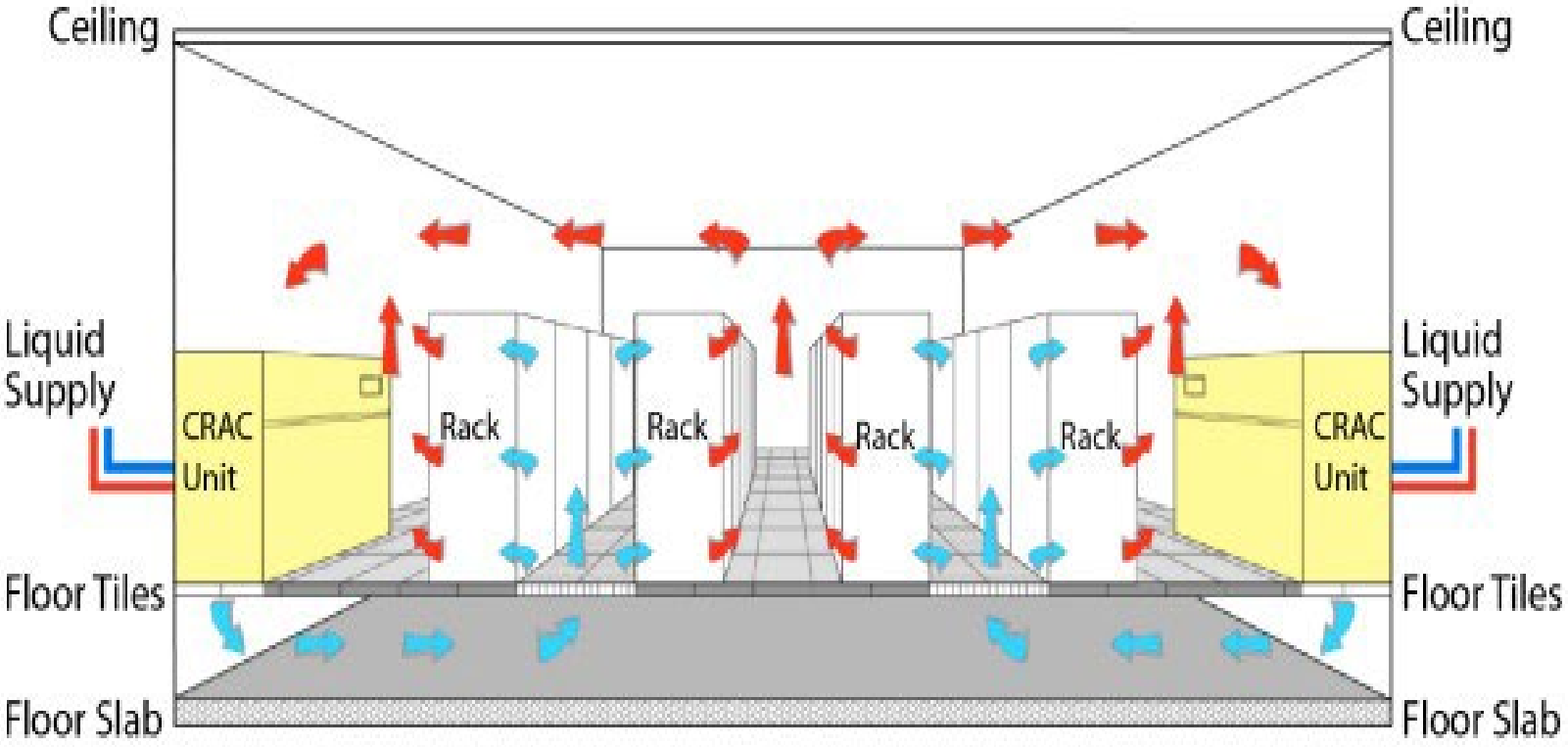
RMIT Classification: Trusted

Anatomy of a Datacenter



Source: Barroso and Urs Hölzle (2013)

Anatomy of a Datacenter



Source: Barroso and Urs Hölzle (2013)



Source: Google

Energy Consumed by Data Centres

- U.S. data centres
  - use > 90 billion kilowatt-hours of electricity a year
  - requiring roughly 34 giant (500-megawatt) coal-powered plants.
- Global data centres
  - used roughly 416 terawatts (4.16×10^{14} watts) (or about 3% of the total electricity 2016),
  - nearly 40% more than the entire United Kingdom.

The consumption will double every four years.

End of Part 1

Part 2

Apache Hadoop

Apache Hadoop

Over the next five years the Hadoop market will register a 33.0% CAGR (Compound Annual Growth Rate) in terms of revenue, the global market size will reach US$ 9.4 billion by 2024, from US$1.7 billion in 2019.

Source: https://www.marketwatch.com/press-release/at-33-cagr-hadoop-market-size-is-expected-to-exhibit-9400-million-usd-by-2024-2020-07-27?mod=mw_quote_news#:~:text=According%20to%20this%20study%2C%20over,key%20companies%20in%20Hadoop%20business.

Apache Hadoop

- An open source software framework
- For storage and large scale processing of data-sets on clusters of commodity hardware.

Hadoop was created by Doug Cutting & Mike Cafarella in 2005.

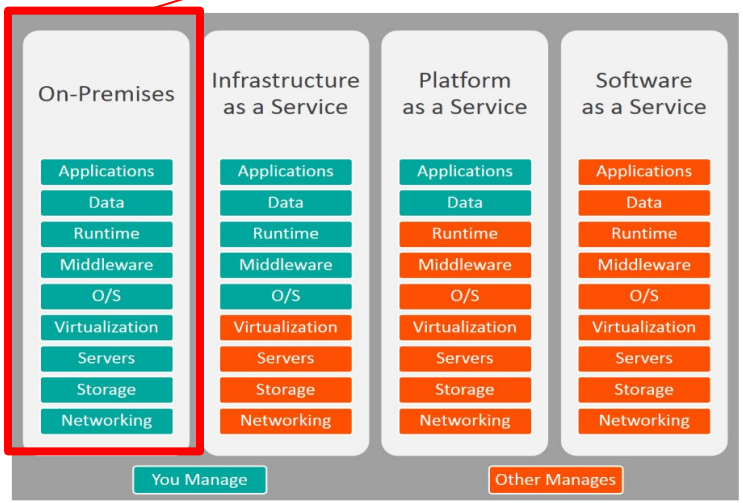
- Originally developed to support distribution for the Nutch search engine project.
- Doug, who was working at Yahoo! named the project after his son's toy elephant.





Apache Hadoop

- **Hadoop on-premise**



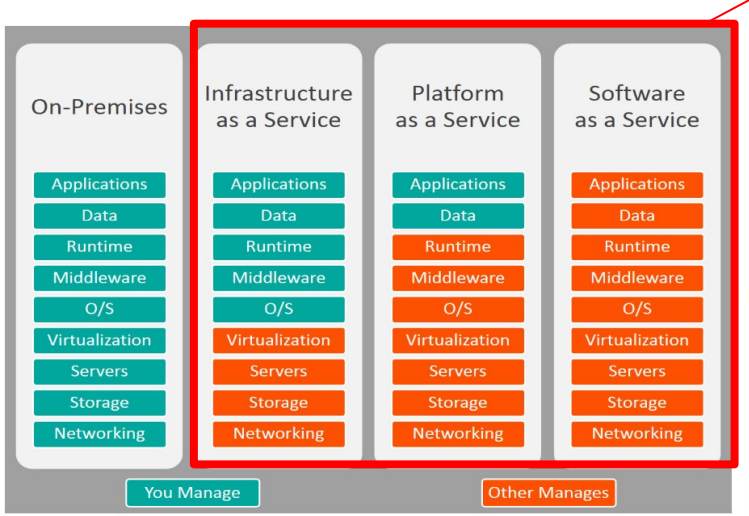


Three biggest and most prevalent Hadoop distributions:

- Cloudera
- Hortonworks
- MapR

Apache Hadoop

- **Hadoop in the Cloud**





Some providers

- AWS EMR
- Microsoft Azure
- Google Cloud Dataproc

…

Apache Hadoop

- **Hadoop in the Cloud**

  Some providers

  – AWS EMR

  Amazon Web Services' solution for managing pre-packaged Hadoop clusters and running jobs on them. You can work with regular MapReduce jobs or Apache Spark jobs, and can use Apache Hive, Apache Pig, Apache HBase, and some third-party applications. Data is typically stored in Amazon S3 or Amazon DynamoDB.



Apache Hadoop

- **Hadoop in the Cloud**

  Some providers

  – Microsoft Azure's pre-packaged solution (called HDInsight)

  built on top of the Hortonworks Data Platform (HDP). The service defines cluster types for Hadoop, Spark, Apache Storm, and HBase; other components like Hive and Pig are included as well.



Apache Hadoop

- **Hadoop in the Cloud**

  Some providers

  – Google Cloud Dataproc

  runs within Google Cloud Platform. It offers Hadoop, Spark, Hive, and Pig, working on data that is usually stored in Google Cloud Storage. Like EMR, it supports both transient and long-running clusters, cluster resizing, and scripts for installing additional services. It can also be controlled through an API.



Apache Hadoop HDFS

The base Apache Hadoop framework is composed of various modules



The Hadoop Distributed File System

Apache Hadoop HDFS

HDFS follows the write-once, read-many approach for its files and applications.

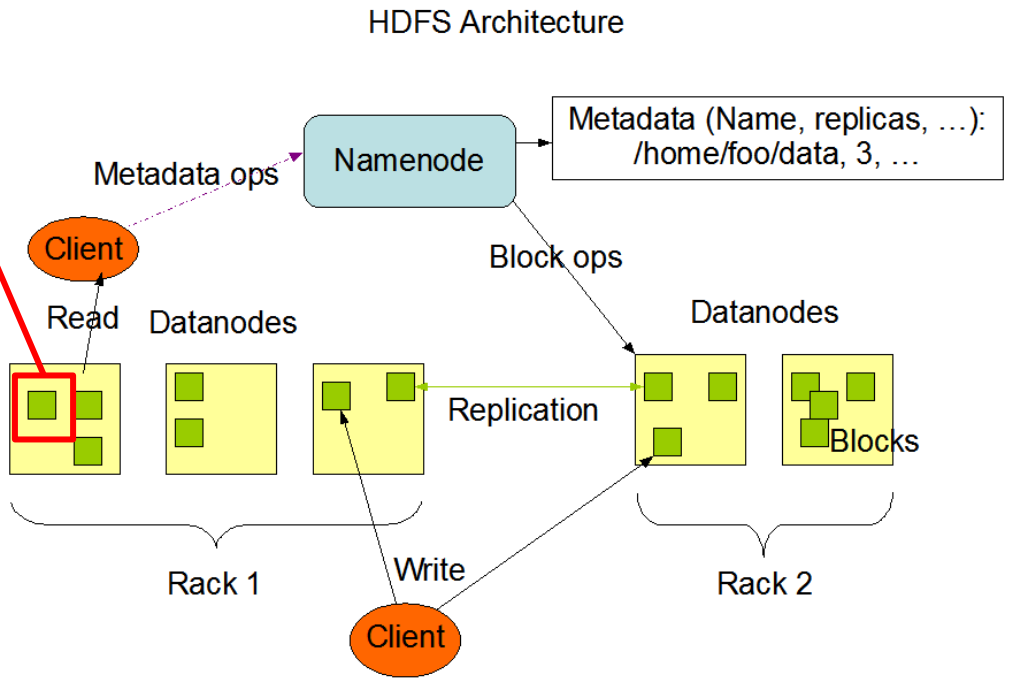
A file in HDFS once written will not be modified, though it can be access 'n' number of times

Apache Hadoop HDFS

- HDFS is a distributed file system
  - HDFS is highly fault-tolerant
    - HDFS is designed to be deployed on low-cost hardware
  - HDFS provides high throughput access to application data
    - Suitable for applications that have large data sets.
  - HDFS allows streaming access to file system data.

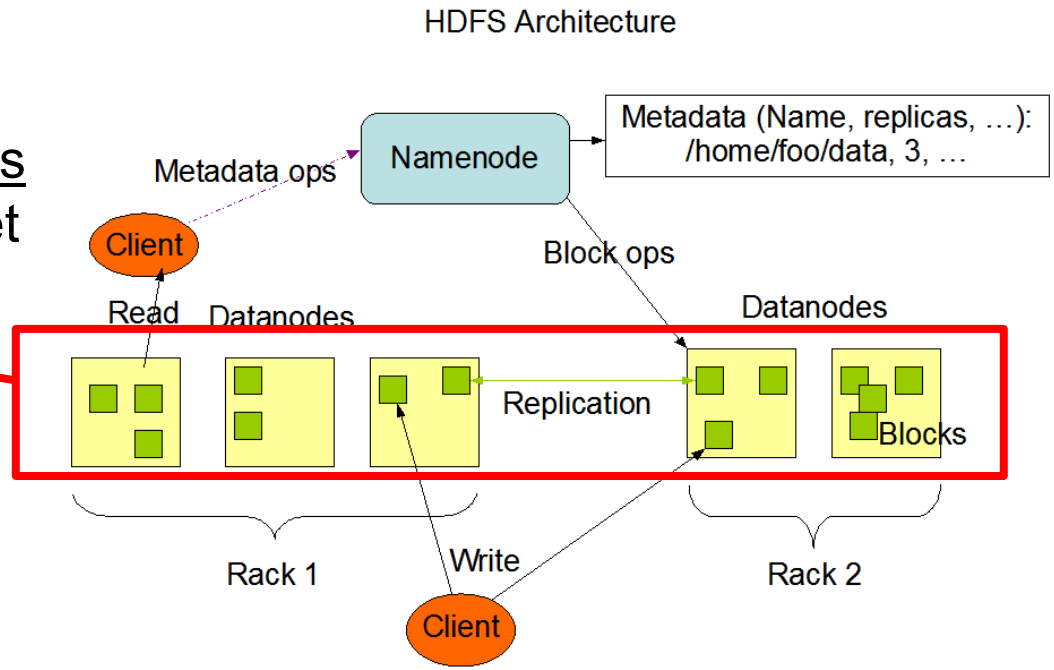
Apache Hadoop HDFS - Architecture.

A file is split into one or more blocks and these blocks are stored in a set of DataNodes



Apache Hadoop HDFS - Architecture.

A file is split into one or more blocks and these blocks are stored in a set of DataNodes



Apache Hadoop HDFS - Architecture.

Master/Slave Architecture.

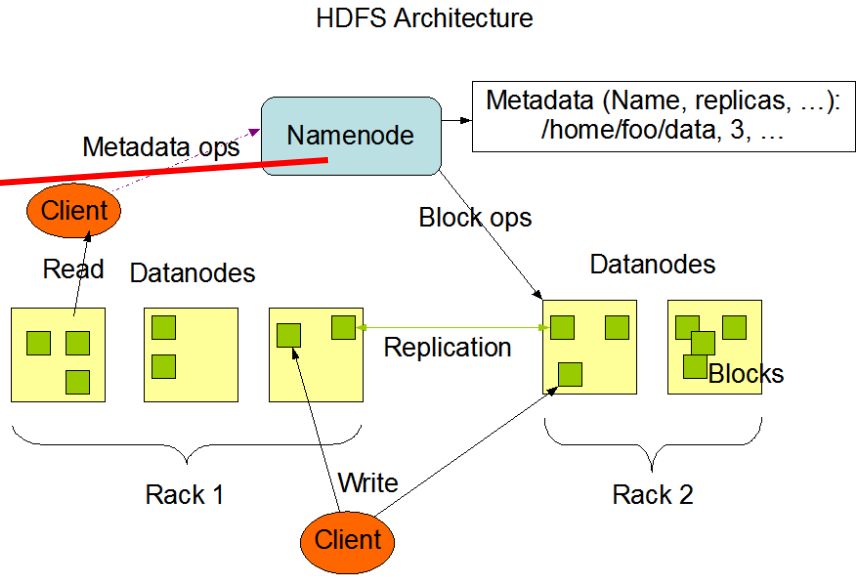
- A Single NameNode

The NameNode maintains the file system namespace.

– Maintain the file system tree and

– Maintain the metadata for all the files

Metadata contains things like the owners of files, permission bits, block location, size etc.



Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.

Apache Hadoop HDFS - Architecture.

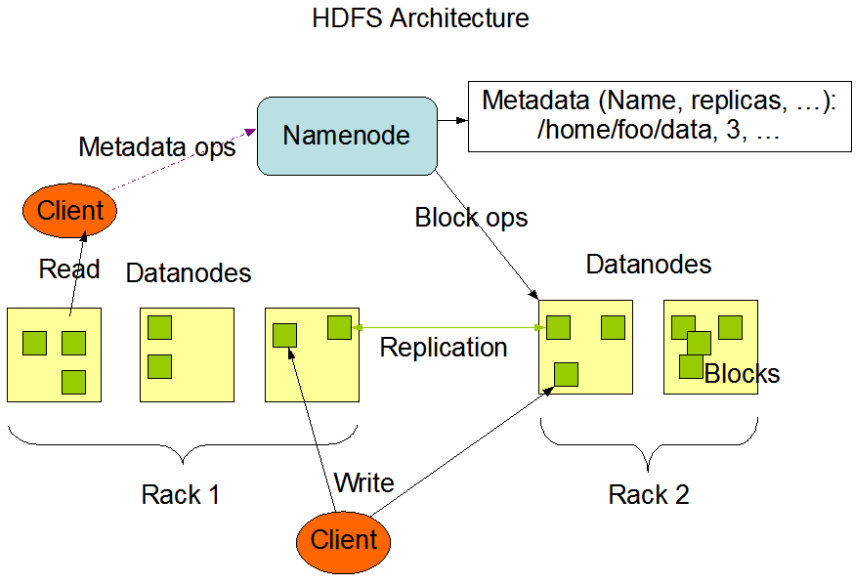
Master/Slave Architecture.

- A Single NameNode

The NameNode maintains the file system namespace.

- Maintain the file system tree and
- Maintain the metadata for all the files

Metadata contains things like the owners of files, permission bits, block location, size etc.



Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.

HDFS supports a traditional hierarchical file organization.

- A user or an application can create directories and store files inside these directories.
- The file system namespace hierarchy is similar to most other existing file systems;
  - one can create and remove files,
  - move a file from one directory to another,
  - rename a file.

Apache Hadoop HDFS - Architecture.

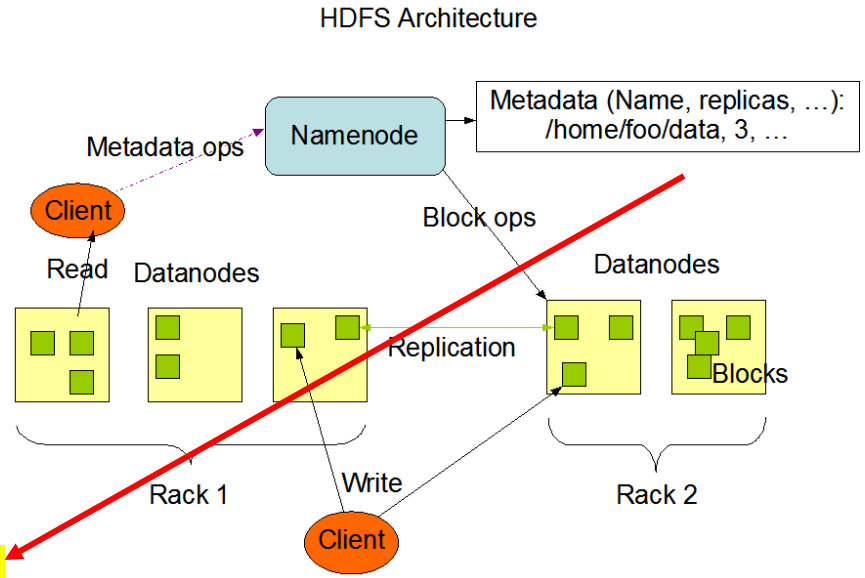
Master/Slave Architecture.

- A Single NameNode

The NameNode maintains the file system namespace.

- Maintain the file system tree and
- Maintain the metadata for all the files

Metadata contains things like the owners of files, permission bits, block location, size etc.



Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.

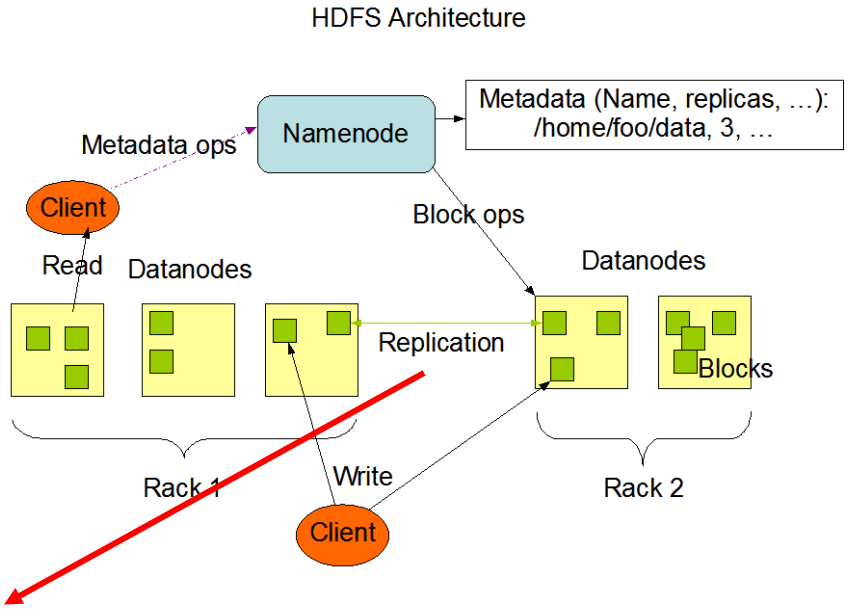
Apache Hadoop HDFS - Architecture.

Master/Slave Architecture.

- A Single NameNode

The NameNode maintains the file system namespace.

- Any change to the file system namespace or its properties is recorded by the NameNode.
- An application can specify the number of replicas of a file that should be maintained by HDFS.
  - The number of copies of a file is called the replication factor of that file.
  - This information is stored by the NameNode.

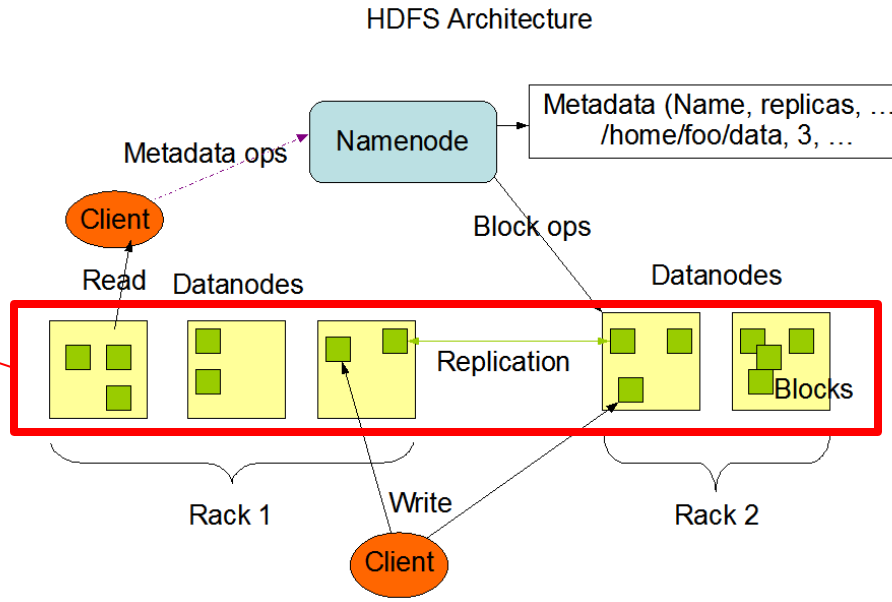


Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.

Apache Hadoop HDFS - Architecture.

Master/Slave Architecture

- A Single NameNode
- A number of DataNodes

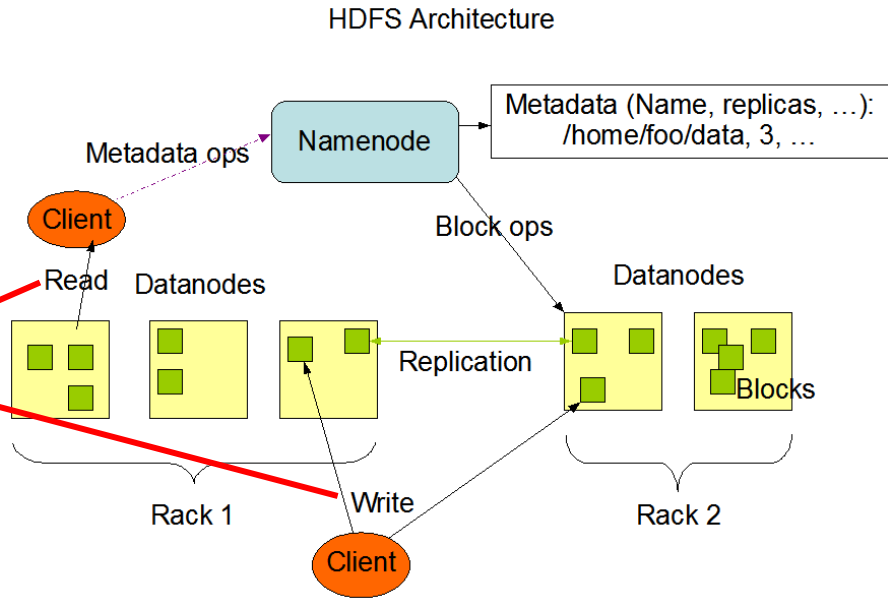


Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.

Apache Hadoop HDFS - Architecture.

Master/Slave Architecture

- A Single NameNode
- A number of DataNodes
  - The DataNodes are responsible for serving read and write requests from the file system's clients.
  - The DataNodes also perform block creation, deletion, and replication upon instruction from the NameNode.

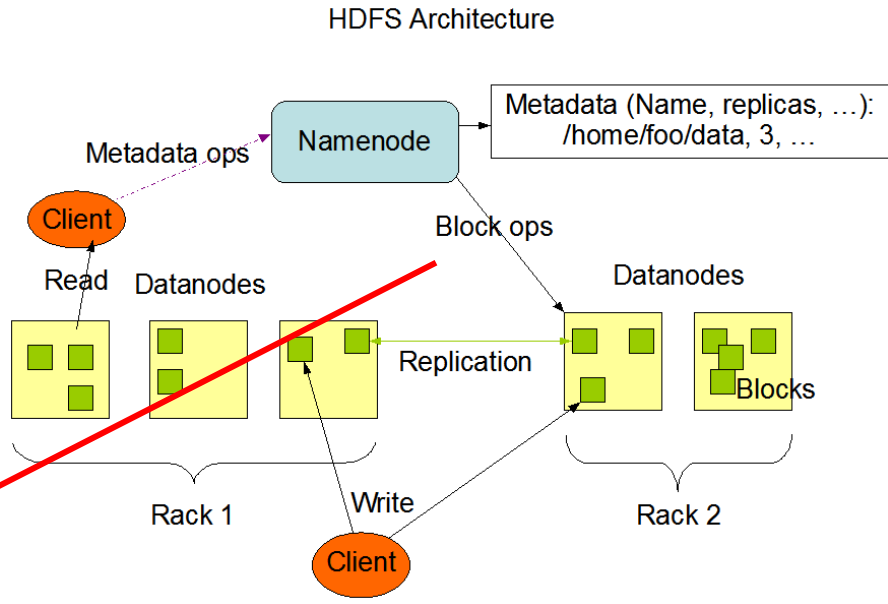


Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.

Apache Hadoop HDFS - Architecture.

Master/Slave Architecture

- A Single NameNode
- A number of DataNodes
  - The DataNodes are responsible for serving read and write requests from the file system's clients.
  - The DataNodes also perform block creation, deletion, and replication upon instruction from the NameNode.

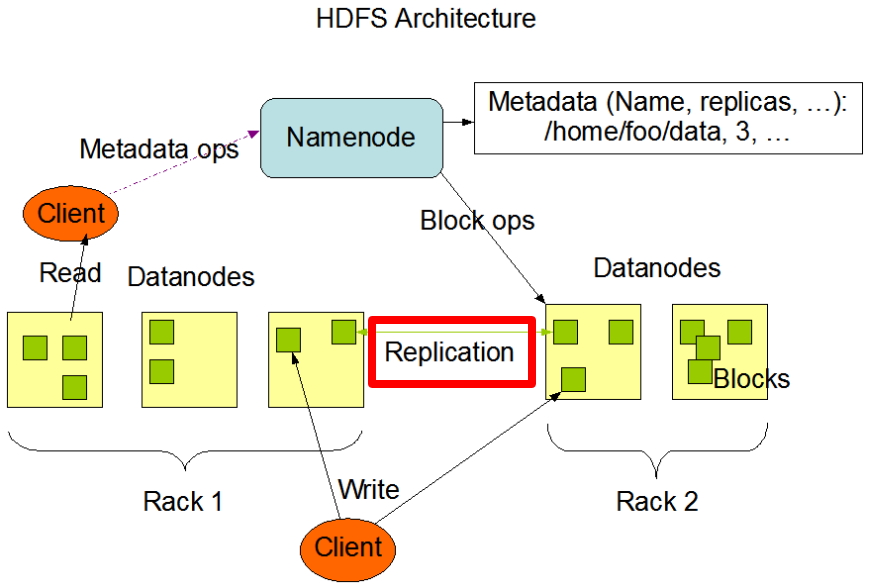


Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.

Apache Hadoop HDFS - Architecture.

- **Data Replication**

HDFS is designed to reliably store very large files across machines in a large cluster.



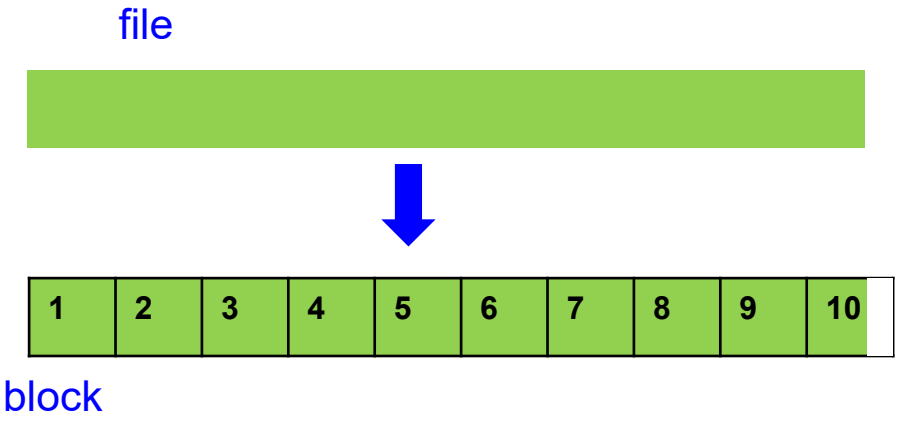
Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.

Apache Hadoop HDFS - Architecture.

- **Data Replication**

HDFS is designed to reliably store very large files across machines in a large cluster.

- It stores each file as a sequence of blocks;
- All blocks in a file except the last block are the same size.
  - The block size and replication factor are configurable per file.
  - An application can specify the number of replicas of a file.
    - The replication factor can be specified at file creation time and can be changed later.

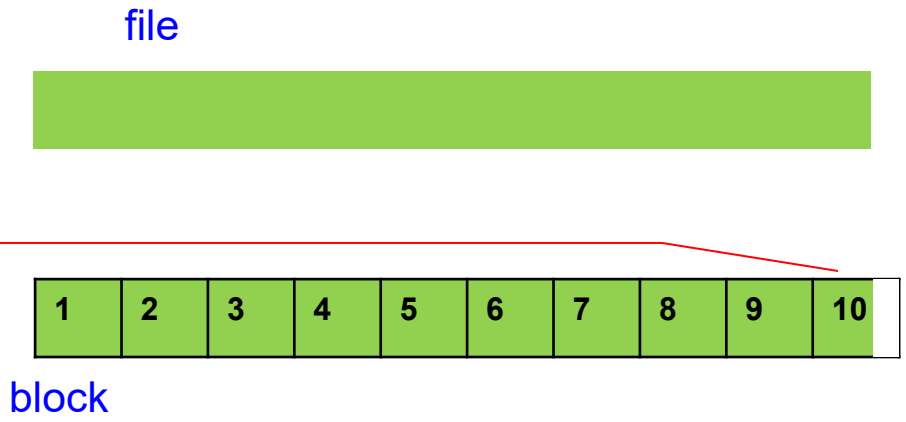


Apache Hadoop HDFS - Architecture.

- **Data Replication**

HDFS is designed to reliably store very large files across machines in a large cluster.

- It stores each file as a sequence of blocks;
- All blocks in a file except the last block are the same size.
  - The block size and replication factor are configurable per file.
  - An application can specify the number of replicas of a file.
    - The replication factor can be specified at file creation time and can be changed later.



Apache Hadoop HDFS - Architecture.

- **Data Replication**

HDFS is designed to reliably store very large files across machines in a large cluster.

- It stores each file as a sequence of blocks;
- All blocks in a file except the last block are the same size.
  - The block size and replication factor are configurable per file.
  - An application can specify the number of replicas of a file.
    - The replication factor can be specified at file creation time and can be changed later.

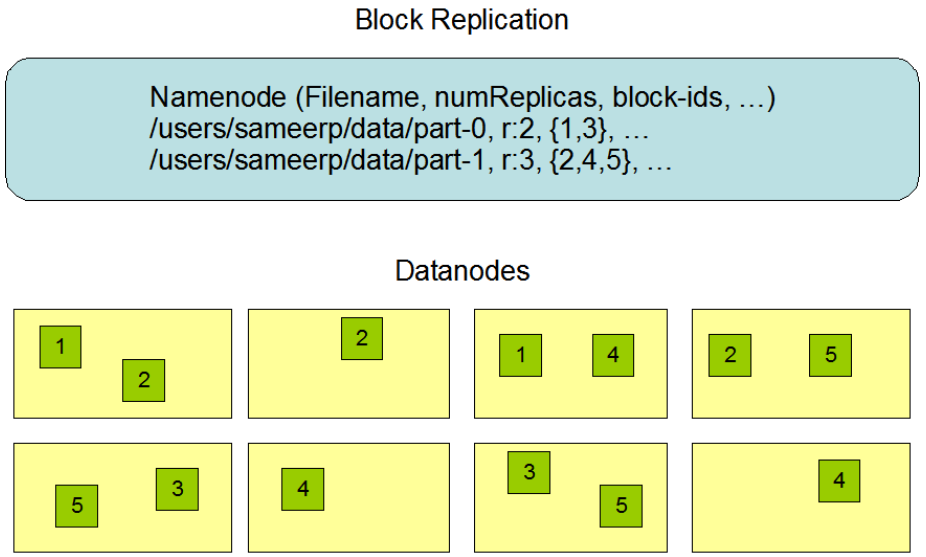


Hadoop Distributed File System (HDFS)

- **Data Replication**

The NameNode manages replication of blocks.

– It periodically receives a Heartbeat and a Blockreport from each of the DataNodes in the cluster.

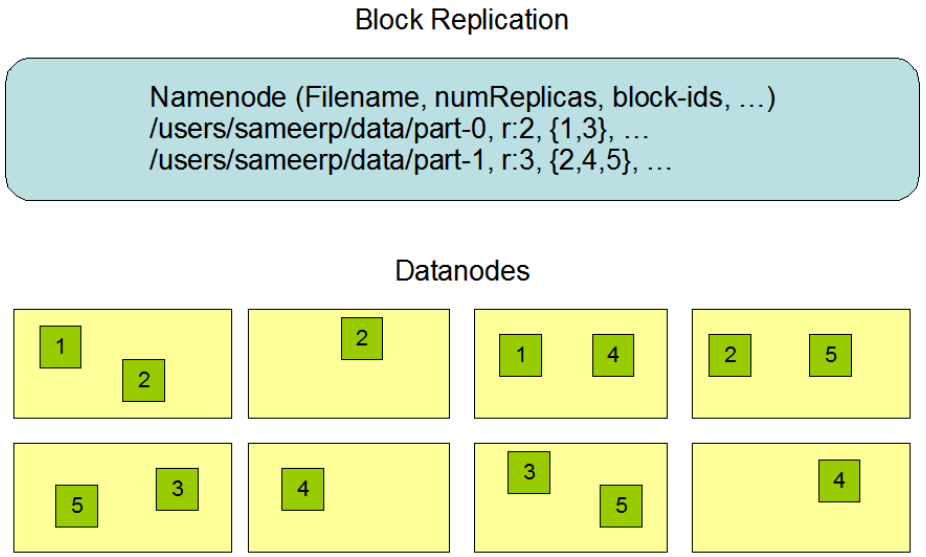


Hadoop Distributed File System (HDFS)

- **Data Replication**

The NameNode manages replication of blocks.

– It periodically receives a Heartbeat and a Blockreport from each of the DataNodes in the cluster.

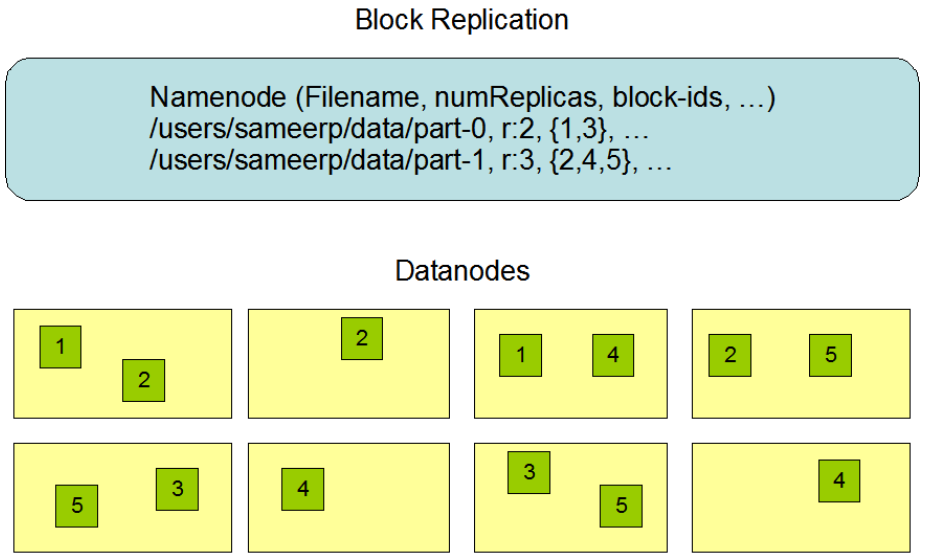


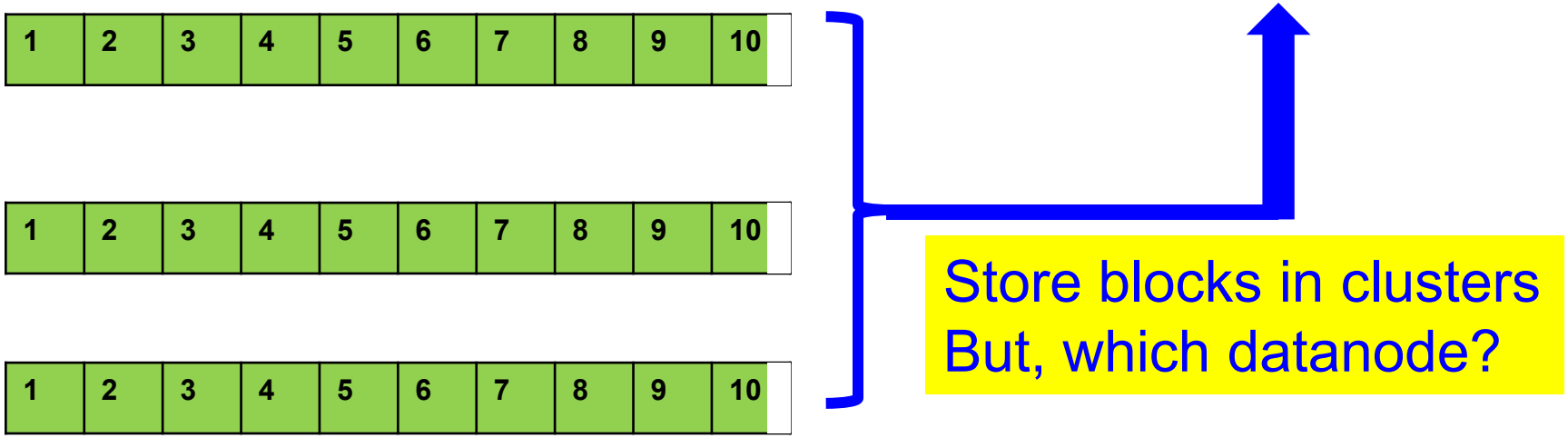
A Blockreport contains a list of all blocks on a DataNode.

Receipt of a Heartbeat implies that the DataNode is functioning properly.

Hadoop Distributed File System (HDFS)

- **Data Replication - Replica Placement**

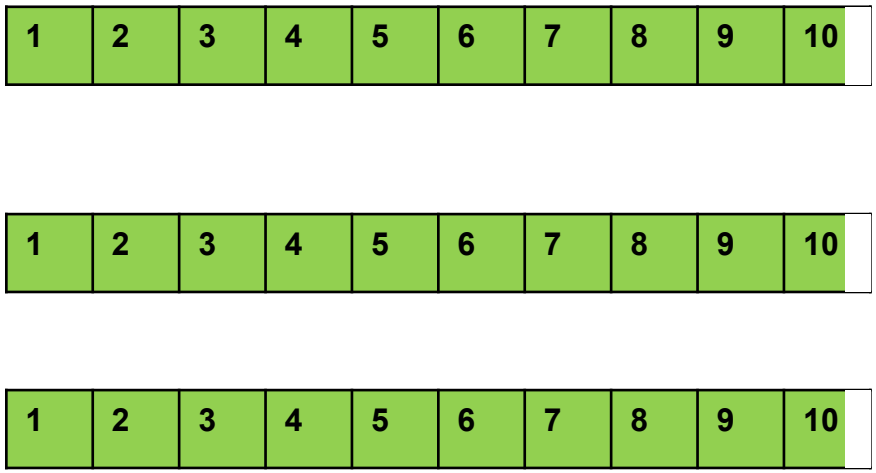


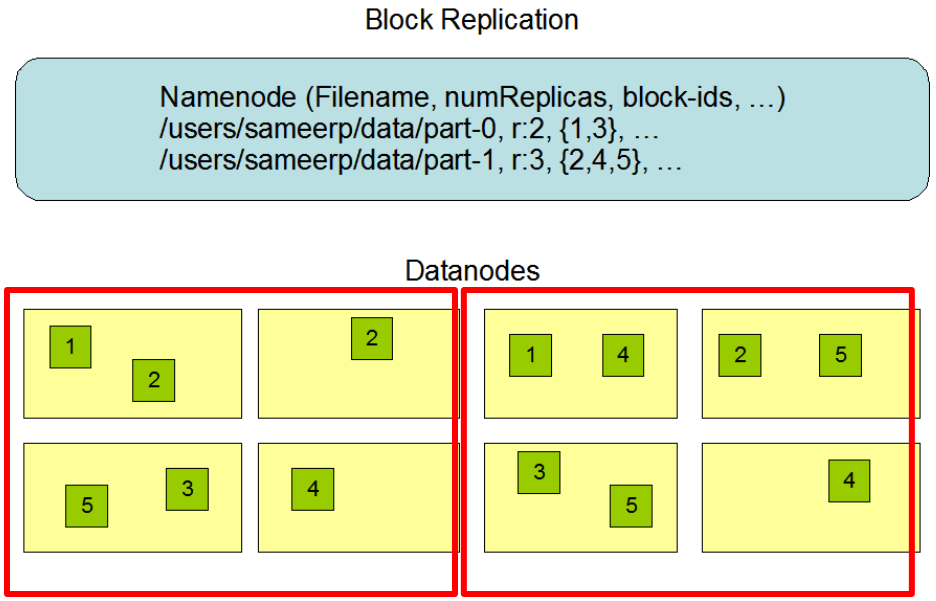


Hadoop Distributed File System (HDFS)

- **Data Replication - Replica Placement**

Large HDFS instances run on a cluster of computers that commonly spread across many racks.





Store blocks in clusters
But, which datanode?

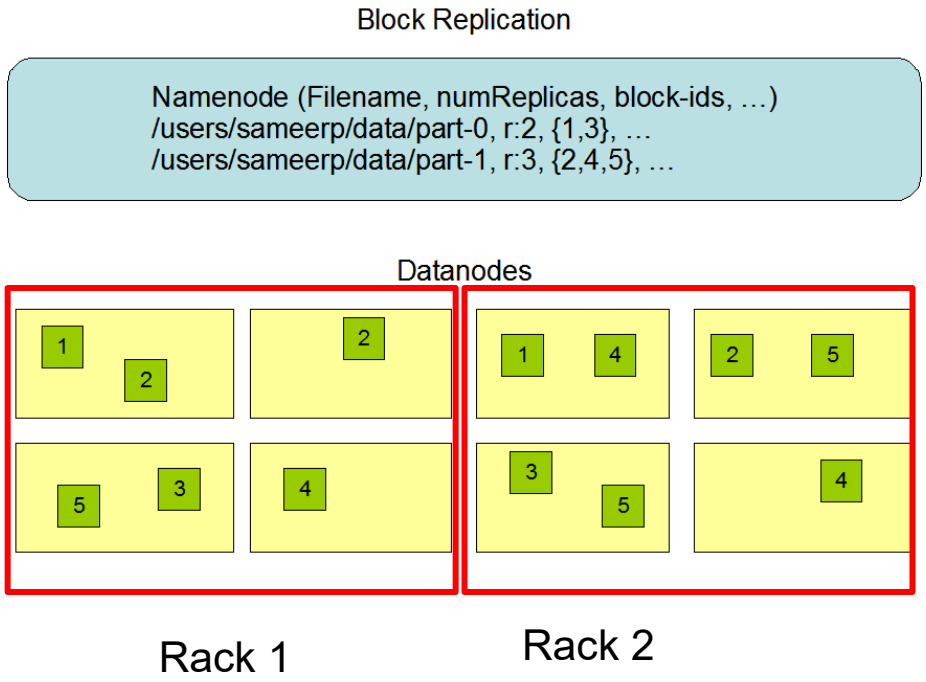
Hadoop Distributed File System (HDFS)

- **Data Replication - Replica Placement**

  Large HDFS instances run on a cluster of computers that commonly spread across many racks.

  – Communication between two nodes in different racks has to go through switches.

    – In most cases, network bandwidth between machines in the same rack is greater than network bandwidth between machines in different racks.



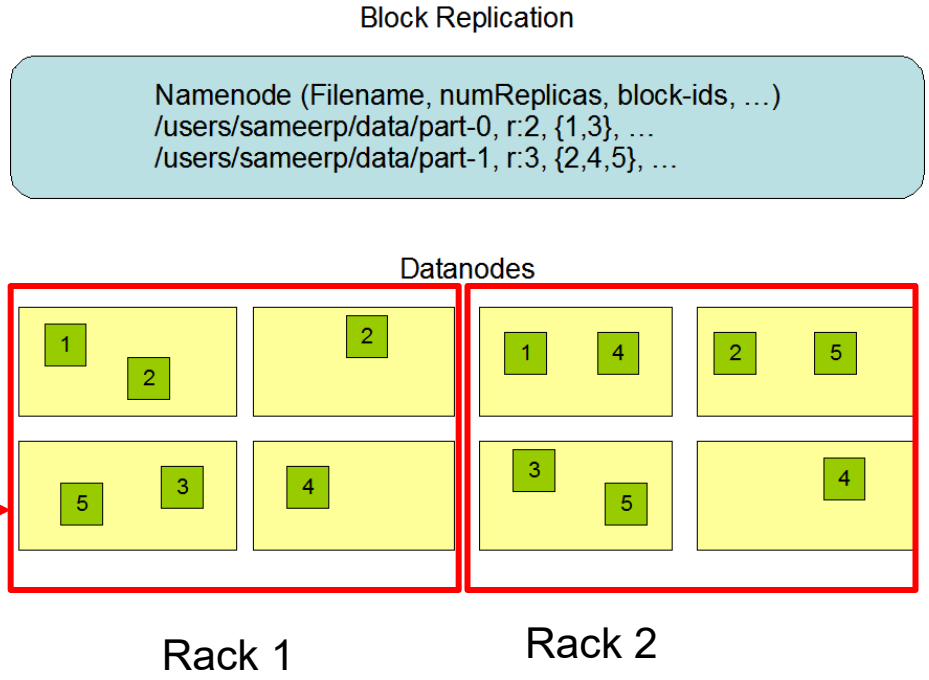
RMIT Classification: Trusted

Hadoop Distributed File System (HDFS)

- **Data Replication - Replica Placement**

  Large HDFS instances run on a cluster of computers that commonly spread across many racks.

  – Communication between two nodes in different racks has to go through switches.
    – In most cases, network bandwidth between machines in the same rack is greater than network bandwidth between machines in different racks.
  – A simple but non-optimal policy is to place replicas on unique racks.
    – Pros
      – This prevents losing data when an entire rack fails and
      – allows use of bandwidth from multiple racks when reading data.
    – Cons
      – However, this policy increases the cost of writes because a write needs to transfer blocks to multiple racks.



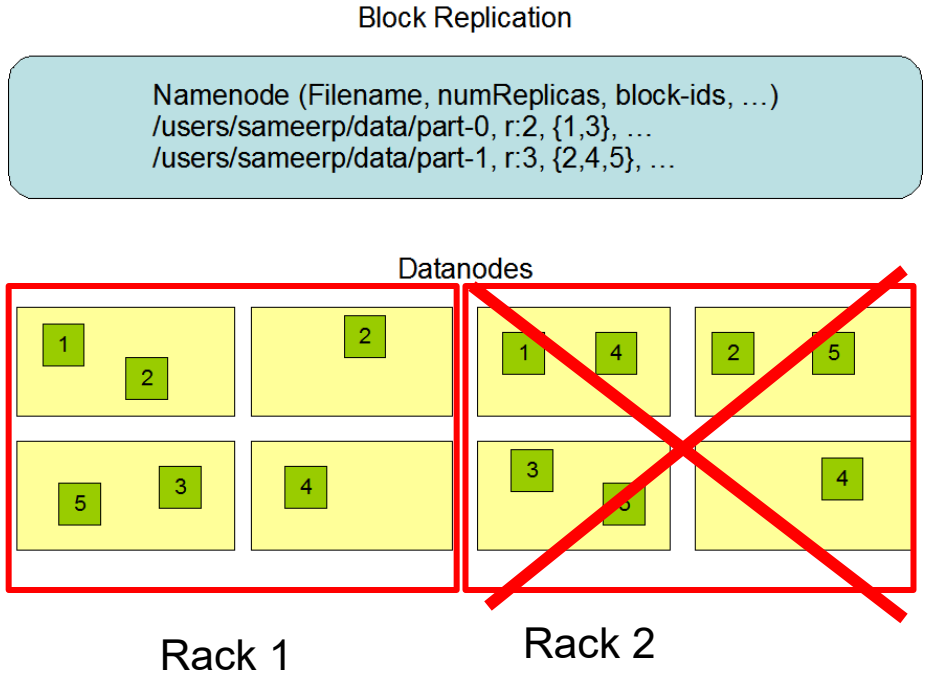
RMIT Classification: Trusted

Hadoop Distributed File System (HDFS)

- **Data Replication - Replica Placement**

  Large HDFS instances run on a cluster of computers that commonly spread across many racks.

  – Communication between two nodes in different racks has to go through switches.
    – In most cases, network bandwidth between machines in the same rack is greater than network bandwidth between machines in different racks.
  – A simple but non-optimal policy is to place replicas on unique racks.
    – Pros
      – This prevents losing data when an entire rack fails and
      – allows use of bandwidth from multiple racks when reading data.
    – Cons
      – However, this policy increases the cost of writes because a write needs to transfer blocks to multiple racks.



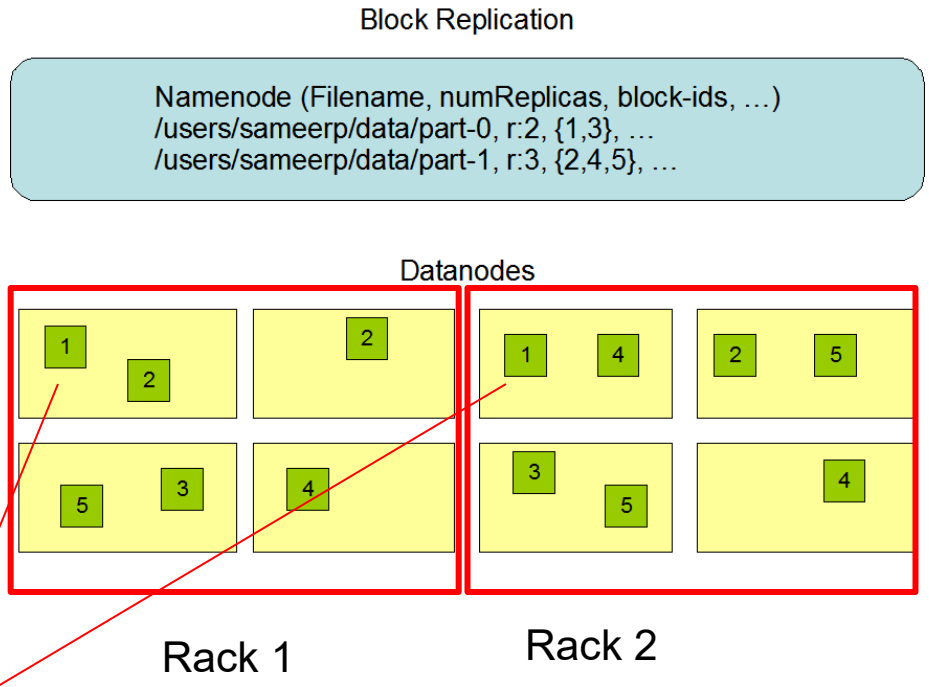
RMIT Classification: Trusted

Hadoop Distributed File System (HDFS)

- **Data Replication - Replica Placement**

  Large HDFS instances run on a cluster of computers that commonly spread across many racks.

  – Communication between two nodes in different racks has to go through switches.
    – In most cases, network bandwidth between machines in the same rack is greater than network bandwidth between machines in different racks.
  – A simple but non-optimal policy is to place replicas on unique racks.
    – Pros
      – This prevents losing data when an entire rack fails and
      – allows use of bandwidth from multiple racks when reading data.
    – Cons
      – However, this policy increases the cost of writes because a write needs to transfer blocks to multiple racks.



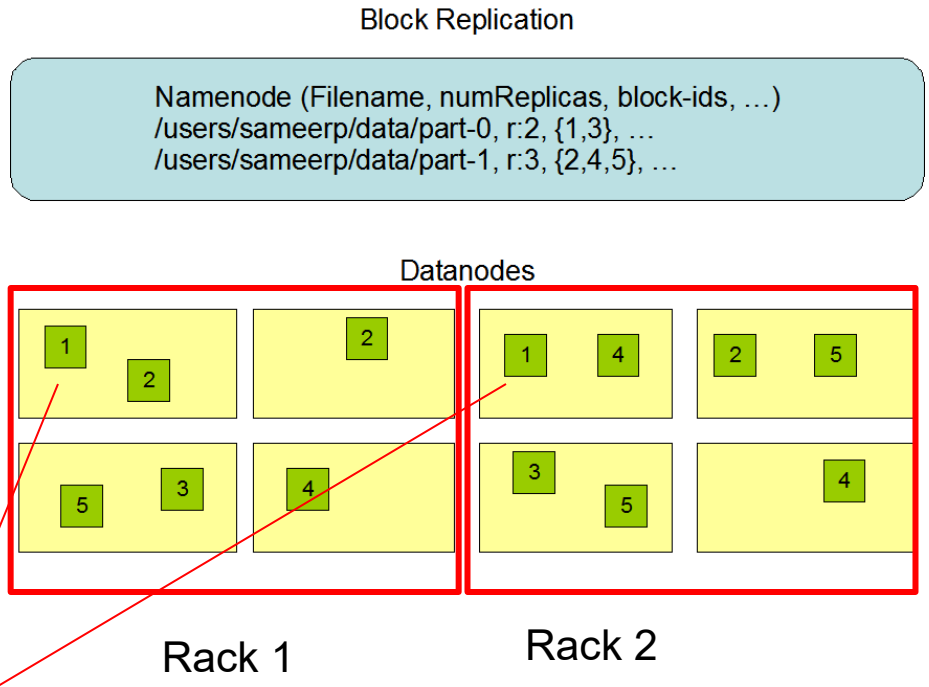
RMIT Classification: Trusted

Hadoop Distributed File System (HDFS)

- **Data Replication - Replica Placement**

  Large HDFS instances run on a cluster of computers that commonly spread across many racks.

  – Communication between two nodes in different racks has to go through switches.
    – In most cases, network bandwidth between machines in the same rack is greater than network bandwidth between machines in different racks.
  – A simple but non-optimal policy is to place replicas on unique racks.
    – Pros
      – This prevents losing data when an entire rack fails and
      – allows use of bandwidth from multiple racks when reading data.
    – Cons
      – However, this policy increases the cost of writes because a write needs to transfer blocks to multiple racks.



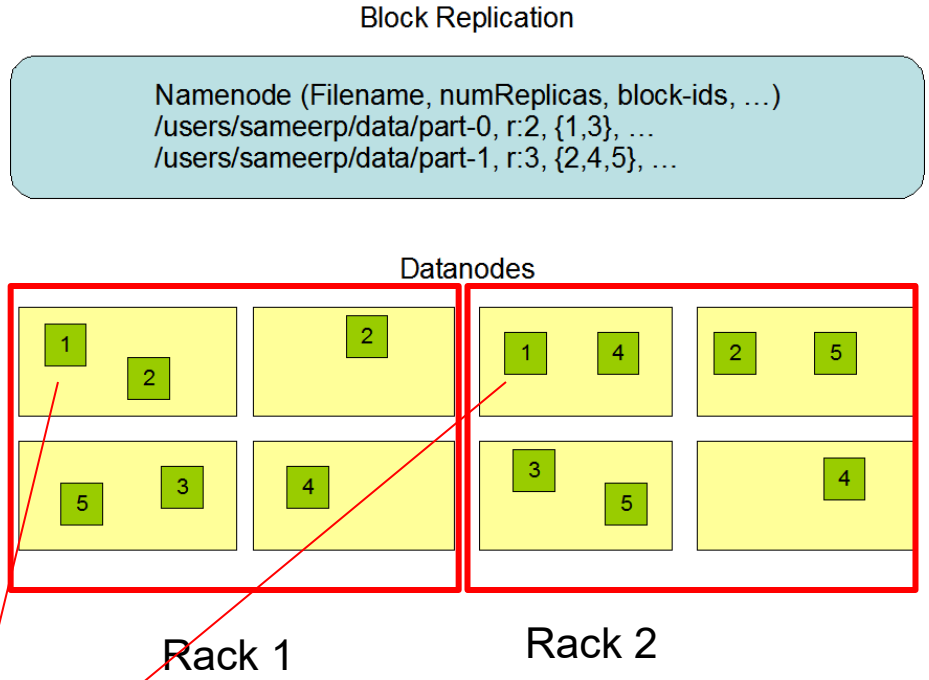
RMIT Classification: Trusted

Hadoop Distributed File System (HDFS)

- **Data Replication - Replica Placement**

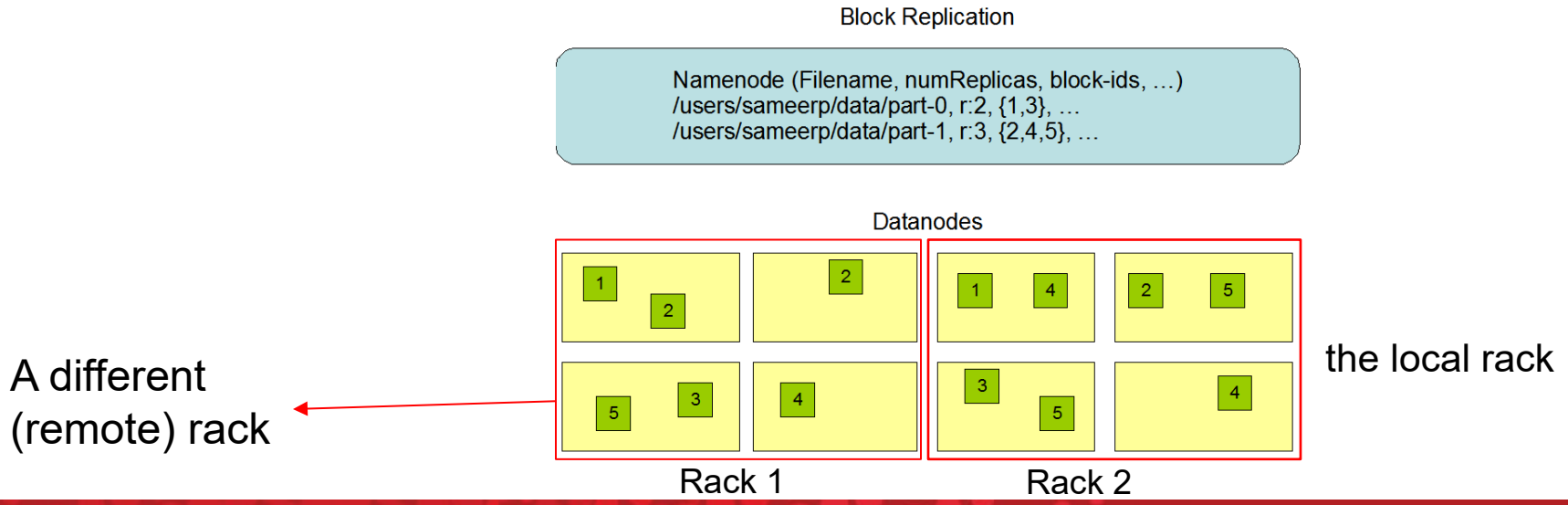
  Large HDFS instances run on a cluster of computers that commonly spread across many racks.

  – Communication between two nodes in different racks has to go through switches.
    – In most cases, network bandwidth between machines in the same rack is greater than network bandwidth between machines in different racks.
  – A simple but non-optimal policy is to place replicas on unique racks.
    – Pros
      – This prevents losing data when an entire rack fails and
      – allows use of bandwidth from multiple racks when reading data.
    – Cons
      – However, this policy increases the cost of writes because a write needs to transfer blocks to multiple racks.



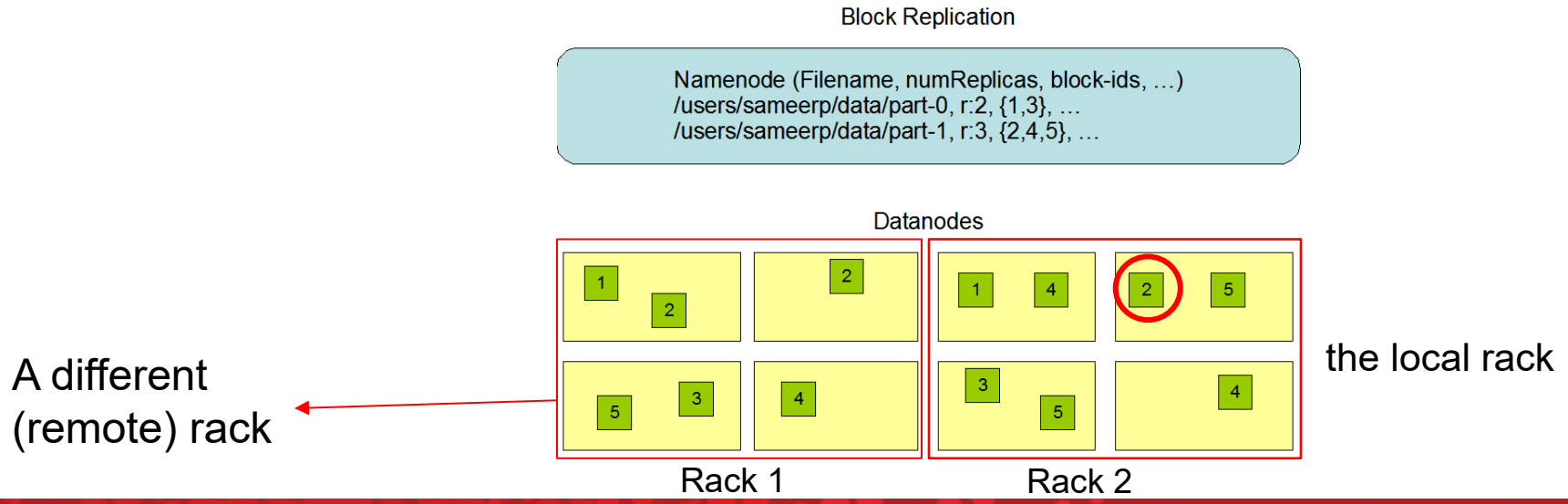
Hadoop Distributed File System (HDFS)

- **Replica Placement Example**
  - Suppose the replication factor is three, HDFS's placement policy is to put
    - one replica on one node in the local rack,
    - another on a node in a different (remote) rack, and
    - the last on a different node in the same remote rack.



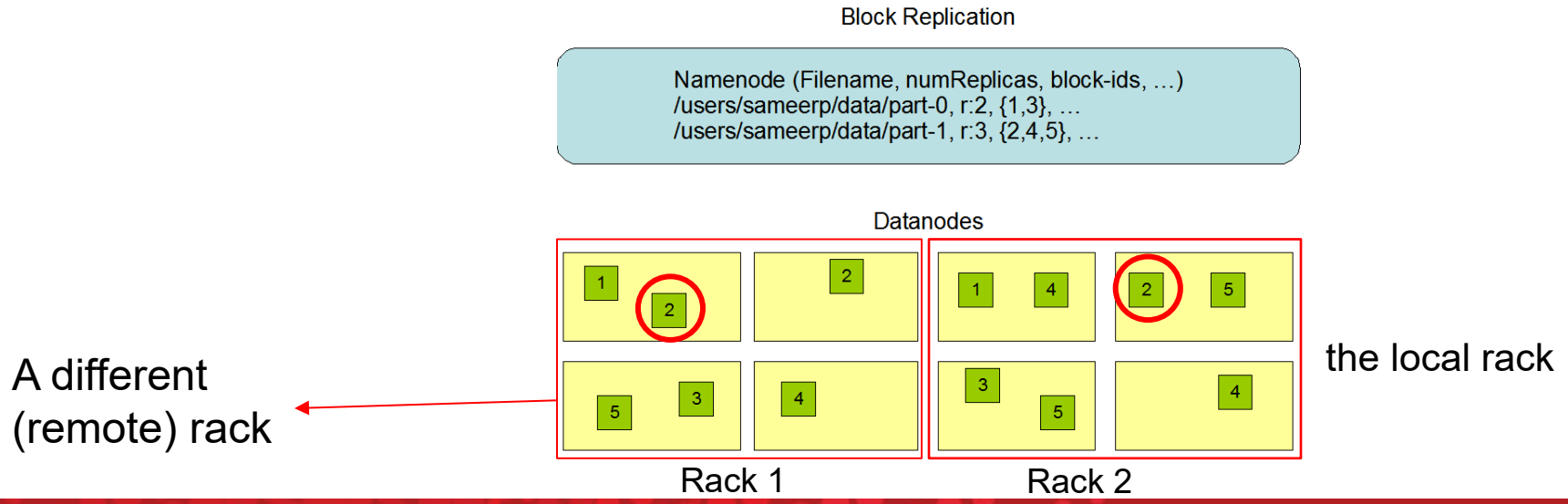
Hadoop Distributed File System (HDFS)

- **Replica Placement Example**
  - Suppose the replication factor is three, HDFS's placement policy is to put
    - one replica on one node in the local rack,
    - another on a node in a different (remote) rack, and
    - the last on a different node in the same remote rack.



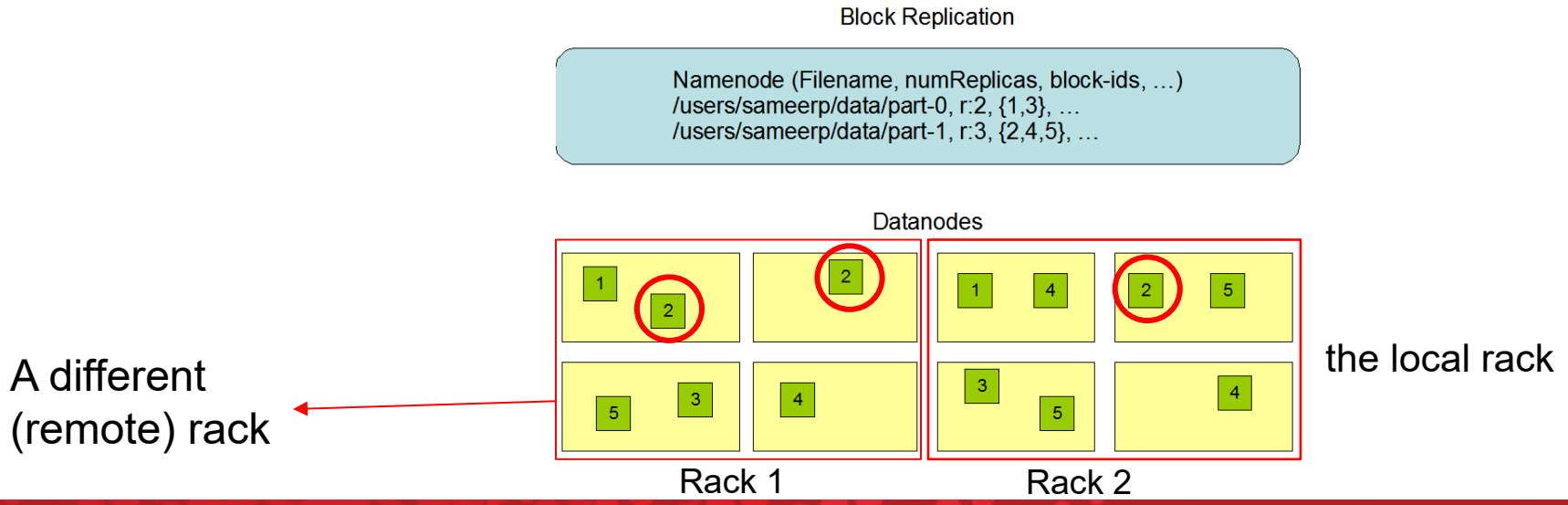
Hadoop Distributed File System (HDFS)

- **Replica Placement Example**
  - Suppose the replication factor is three, HDFS's placement policy is to put
    - one replica on one node in the local rack,
    - another on a node in a different (remote) rack, and
    - the last on a different node in the same remote rack.



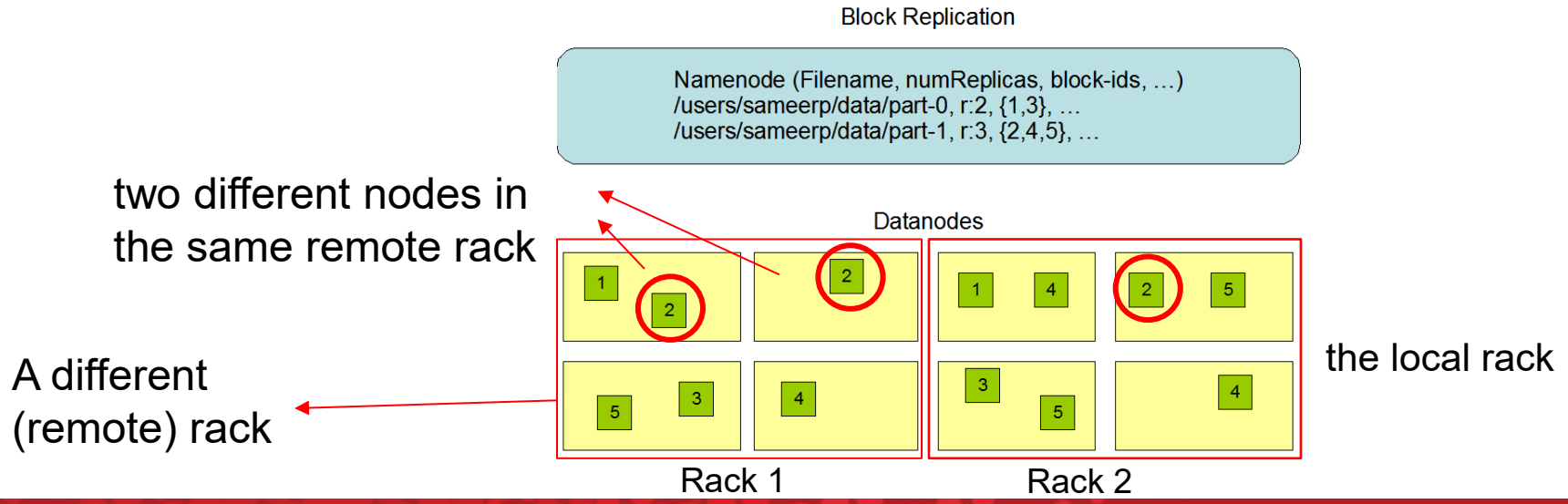
Hadoop Distributed File System (HDFS)

- **Replica Placement Example**
  - Suppose the replication factor is three, HDFS's placement policy is to put
    - one replica on one node in the local rack,
    - another on a node in a different (remote) rack, and
    - the last on a different node in the same remote rack.



Hadoop Distributed File System (HDFS)

- **Replica Placement Example**
  - Suppose the replication factor is three, HDFS's placement policy is to put
    - one replica on one node in the local rack,
    - another on a node in a different (remote) rack, and
    - the last on a different node in the same remote rack.



Hadoop Distributed File System (HDFS)

- **Replica Selection**

To minimize global bandwidth consumption and read latency, HDFS tries to satisfy a read request from a replica that is closest to the reader.

Hadoop Distributed File System (HDFS)

- **Replica Selection**

To minimize global bandwidth consumption and read latency, HDFS tries to satisfy a read request from a replica that is closest to the reader.

- If there exists a replica on the same rack as the reader node, then that replica is preferred to satisfy the read request.
- If a HDFS cluster spans multiple data centers, then a replica that is resident in the local data center is preferred over any remote replica.

End of Part 2

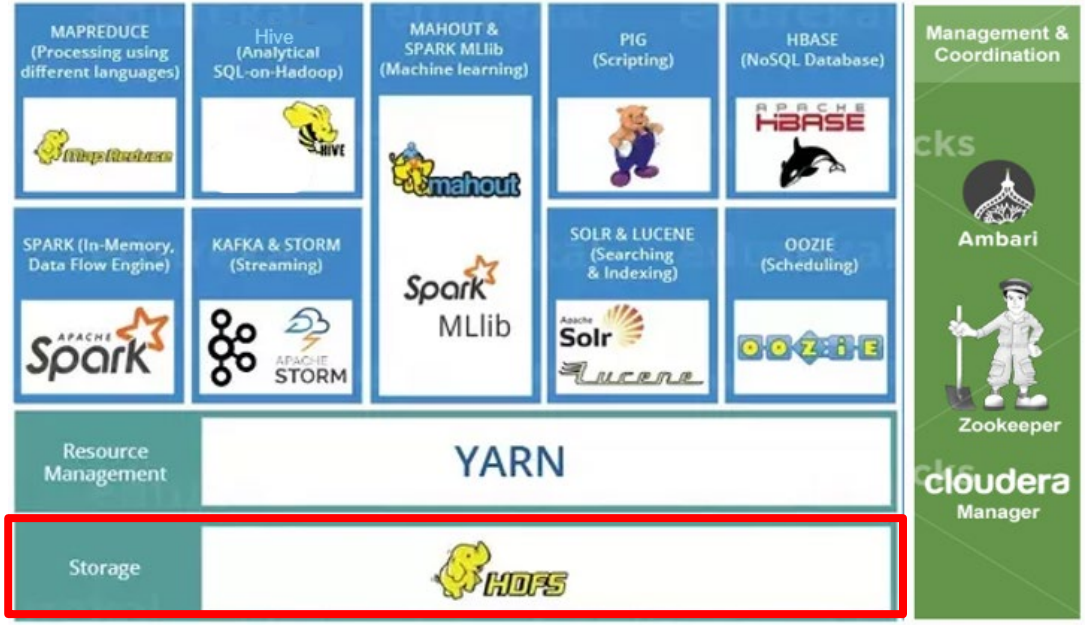
Part 3

Hadoop Yarn

Apache Hadoop Ecosystem

The base Apache Hadoop framework is composed of the following modules:

- **Hadoop Distributed File System (HDFS)** - a distributed file-system that stores data on commodity machines, providing very high aggregate bandwidth across the cluster;



Apache Hadoop Ecosystem

The base Apache Hadoop framework is composed of the following modules:

- **Hadoop Distributed File System (HDFS)** - a distributed file-system that stores data on commodity machines, providing very high aggregate bandwidth across the cluster;
- **Hadoop YARN** - a platform responsible for managing computing resources in clusters and using them for scheduling users' applications;



Apache Hadoop Ecosystem

The base Apache Hadoop framework is composed of the following modules:

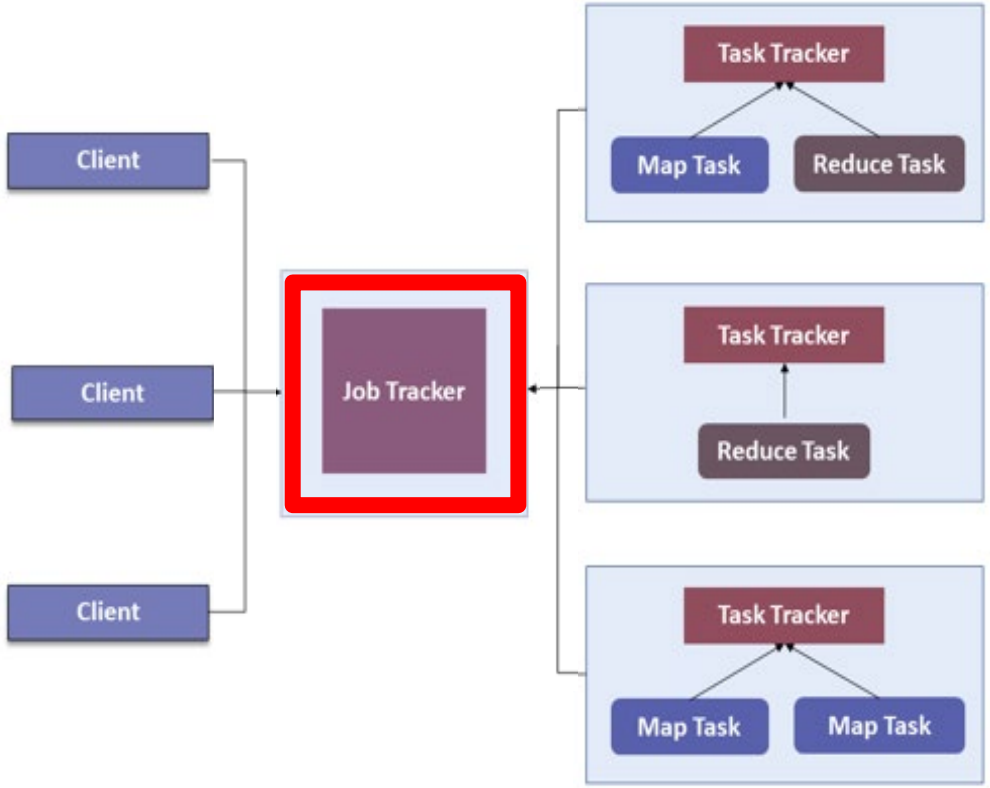
- **Hadoop Distributed File System (HDFS)** - a distributed file-system that stores data on commodity machines, providing very high aggregate bandwidth across the cluster;
- **Hadoop YARN** - a platform responsible for managing computing resources in clusters and using them for scheduling users' applications;



Hadoop 1.0

In Hadoop version 1.0 which is also referred to as MRV1(MapReduce Version 1), MapReduce performed both processing and resource management functions

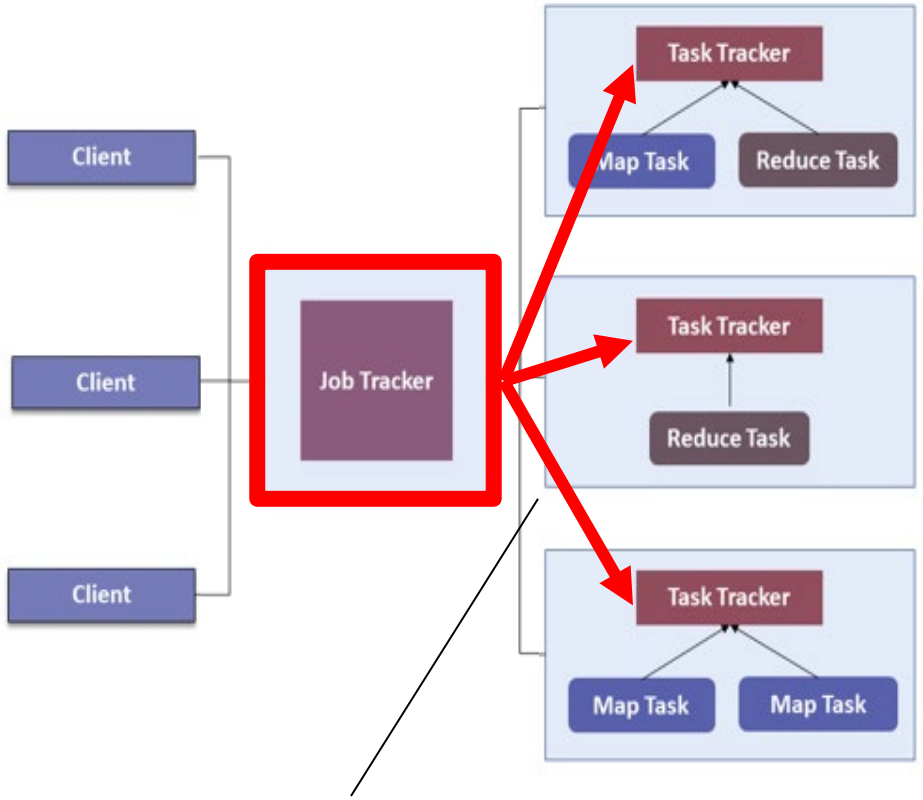
- **Job Tracker - the single master.**
  - Allocate the resources
  - Perform scheduling
  - Monitor the processing jobs



Hadoop 1.0

In Hadoop version 1.0 which is also referred to as MRV1(MapReduce Version 1), MapReduce performed both processing and resource management functions

- **Job Tracker - the single master.**
  - Allocate the resources
  - Perform scheduling
  - Monitor the processing jobs
- **Job Tracker** assign map and reduce tasks on a number of subordinate processes called the Task Trackers.

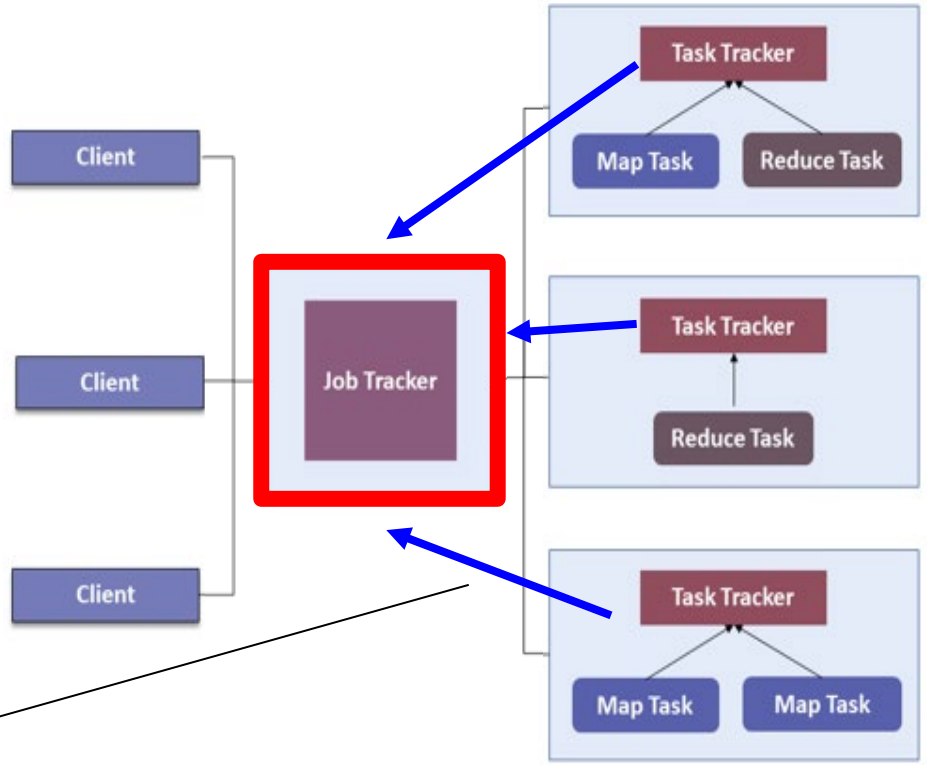


Assign map and reduce tasks

Hadoop 1.0

In Hadoop version 1.0 which is also referred to as MRV1(MapReduce Version 1), MapReduce performed both processing and resource management functions

- **Job Tracker - the single master.**
  - Allocate the resources
  - Perform scheduling
  - Monitor the processing jobs
- **Job Tracker** assign map and reduce tasks on a number of subordinate processes called the Task Trackers.

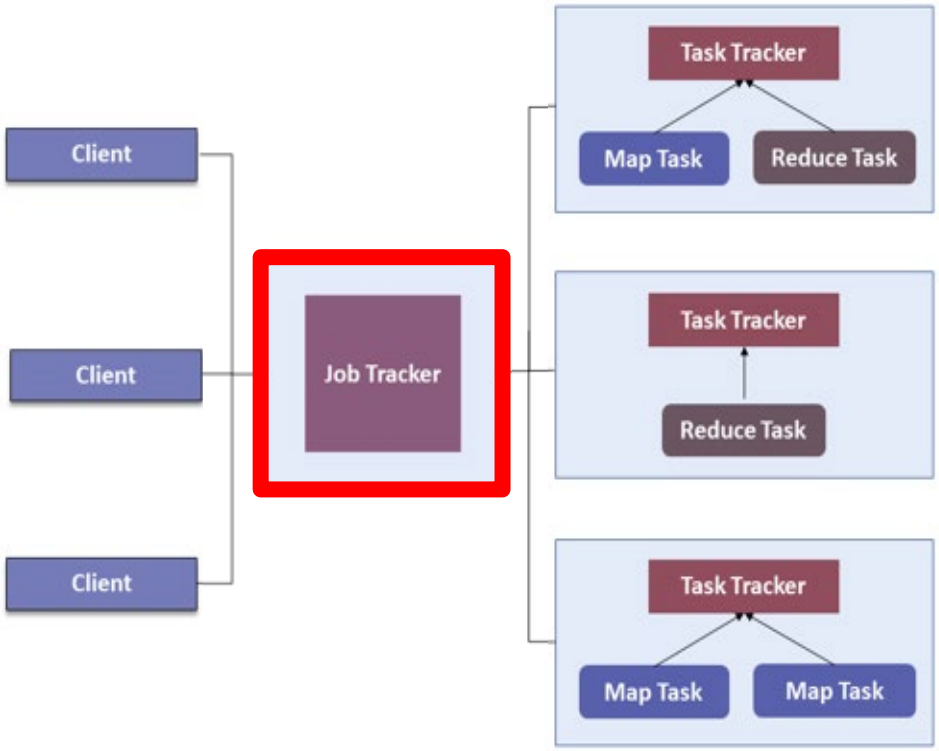


The Task Trackers periodically report their progress to the Job Tracker.

Hadoop 1.0 - Issues

This design resulted in

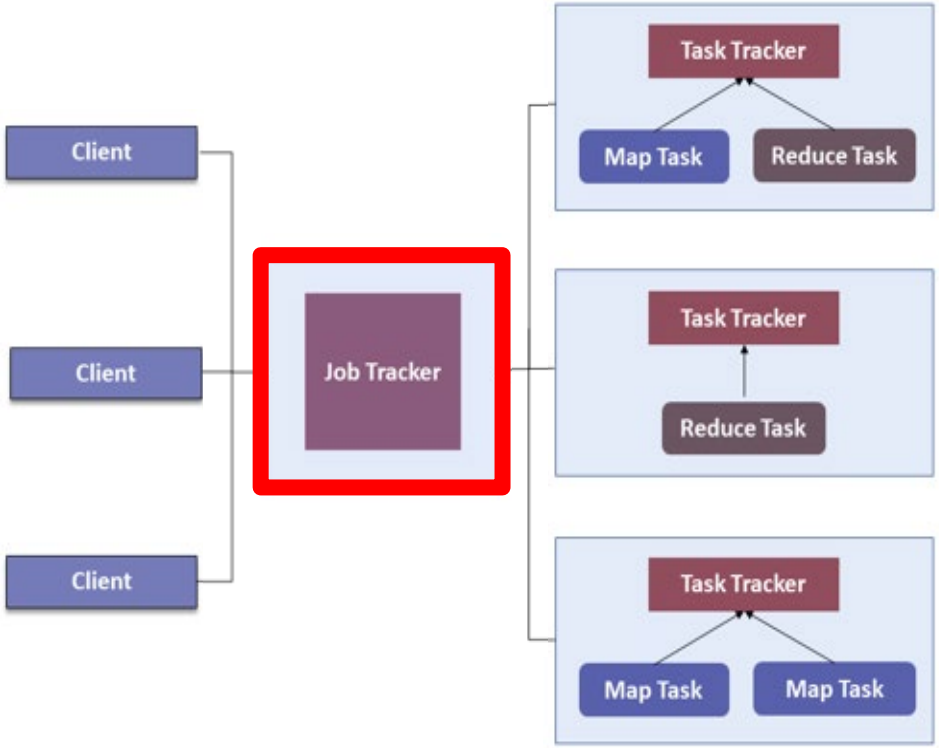
1. Scalability bottleneck due to a single Job Tracker.
   - IBM mentioned in its article that according to Yahoo!, the practical limits of such a design are reached with a cluster of 5000 nodes and 40,000 tasks running concurrently.



Hadoop 1.0 - Issues

This design resulted in

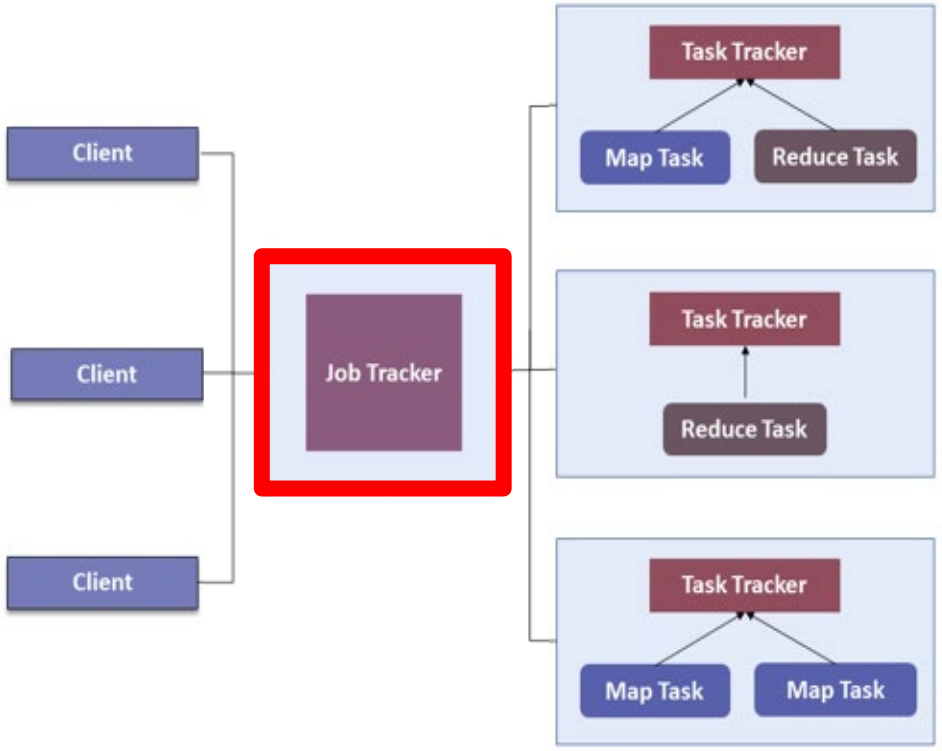
1. Scalability bottleneck due to a single Job Tracker.
   - IBM mentioned in its article that according to Yahoo!, the practical limits of such a design are reached with a cluster of 5000 nodes and 40,000 tasks running concurrently.
2. The Hadoop framework became limited only to MapReduce processing paradigm.



Hadoop 1.0 - Issues

This design resulted in

1. Scalability bottleneck due to a single Job Tracker.
   - IBM mentioned in its article that according to Yahoo!, the practical limits of such a design are reached with a cluster of 5000 nodes and 40,000 tasks running concurrently.
2. The Hadoop framework became limited only to



To overcome all these issues, YARN was introduced in Hadoop version 2.0 in the year 2012 by Yahoo and Hortonworks.

Hadoop YARN



Hadoop v1.0

MapReduce
Data Processing
& Resource Management

HDFS
Distributed File Storage



Hadoop v2.0

MapReduce

Other Data Processing Frameworks

YARN
Resource Management

HDFS
Distributed File Storage

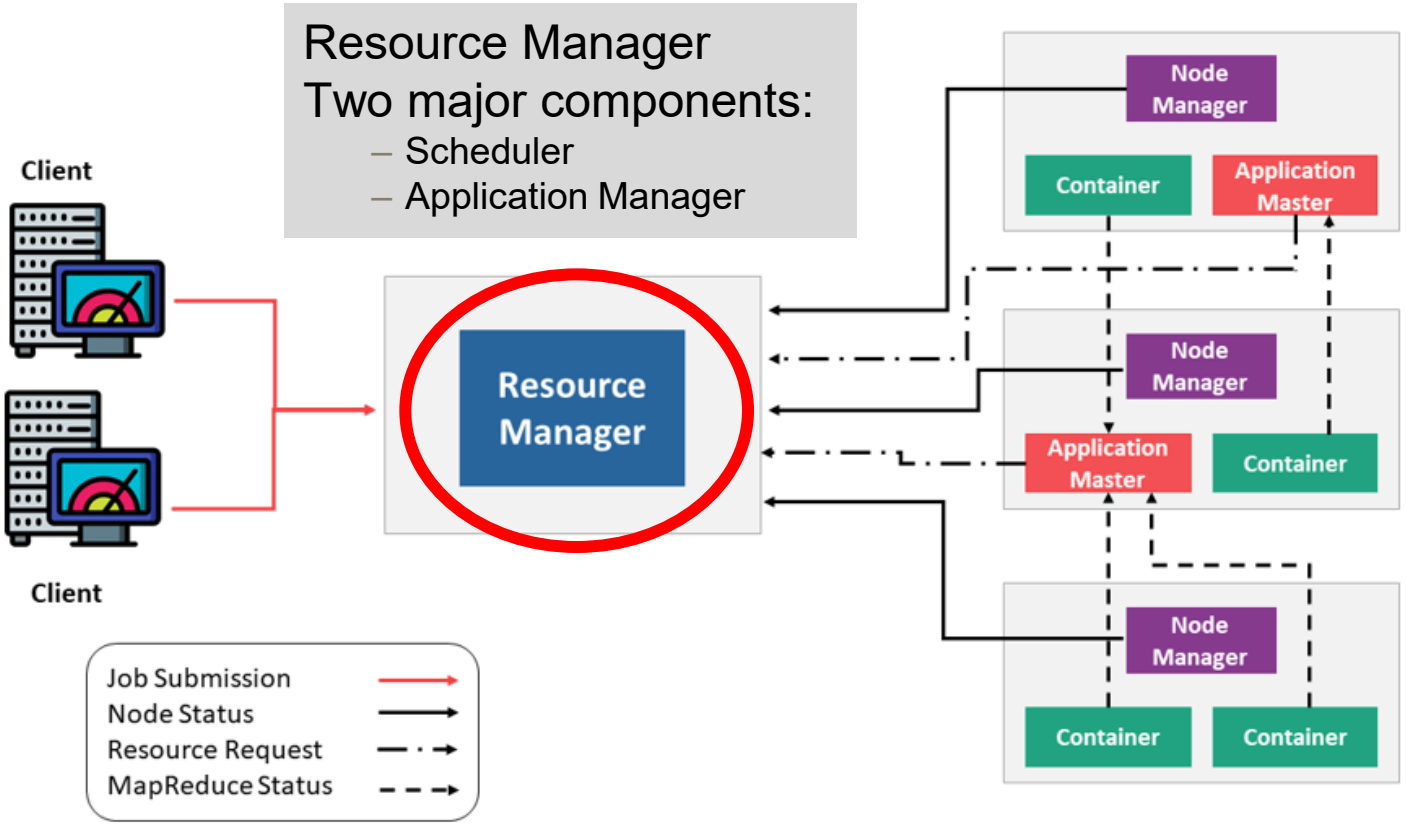
Hadoop YARN (Yet Another Resource Negotiator)

- The basic idea behind YARN is to relieve MapReduce by taking over the responsibility of Resource Management and Job Scheduling.
- With the introduction of YARN, the Hadoop ecosystem became much more flexible, efficient and scalable.
  - When Yahoo went live with YARN in the first quarter of 2013, it aided the company to shrink the size of its Hadoop cluster from 40,000 nodes to 32,000 nodes. But the number of jobs doubled to 26 million per month.

Hadoop YARN

Resource Manager

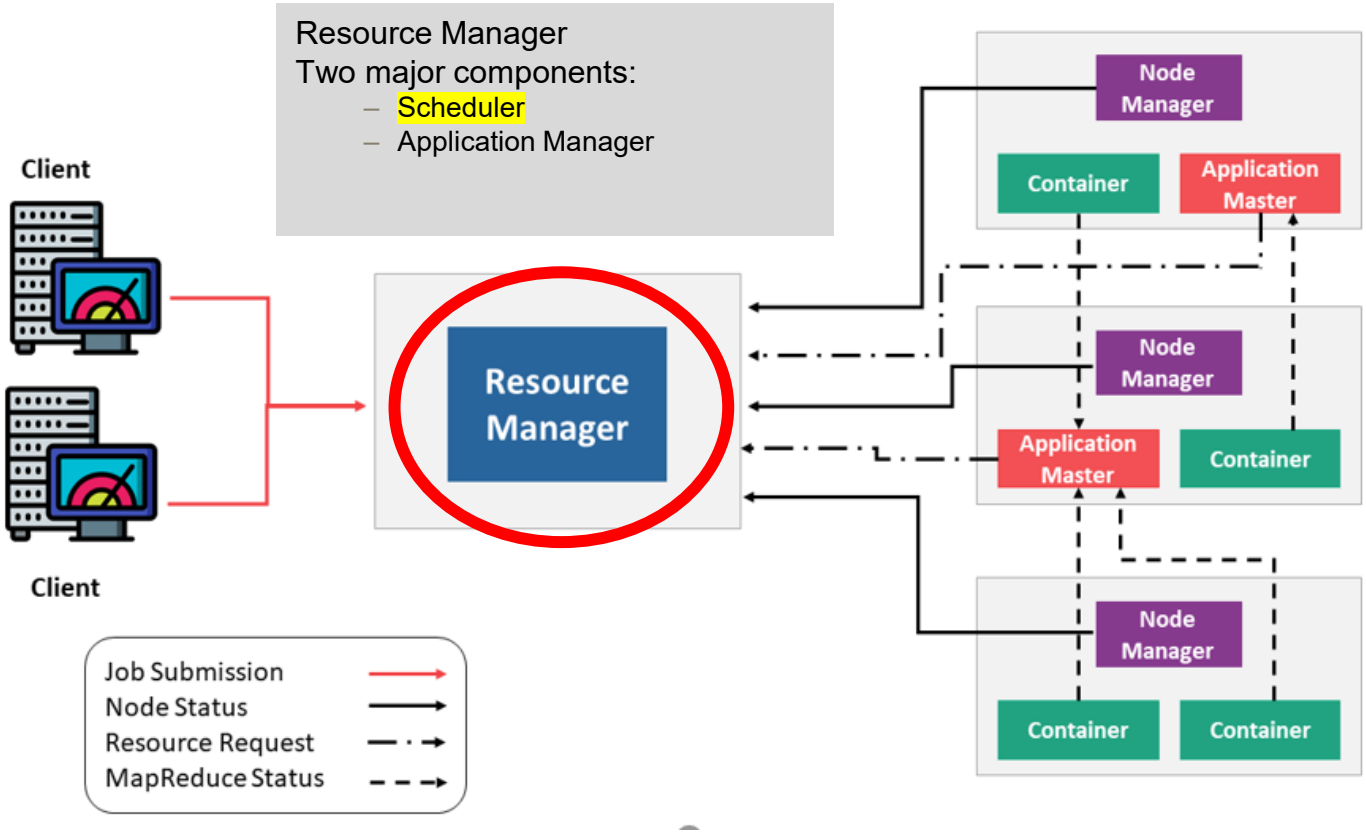
- Arbitrator of the cluster resources
- Decides the allocation of the available resources for competing applications



RMIT Classification: Trusted

Hadoop YARN

Resource Manager
Two major components:
- Scheduler
- Application Manager

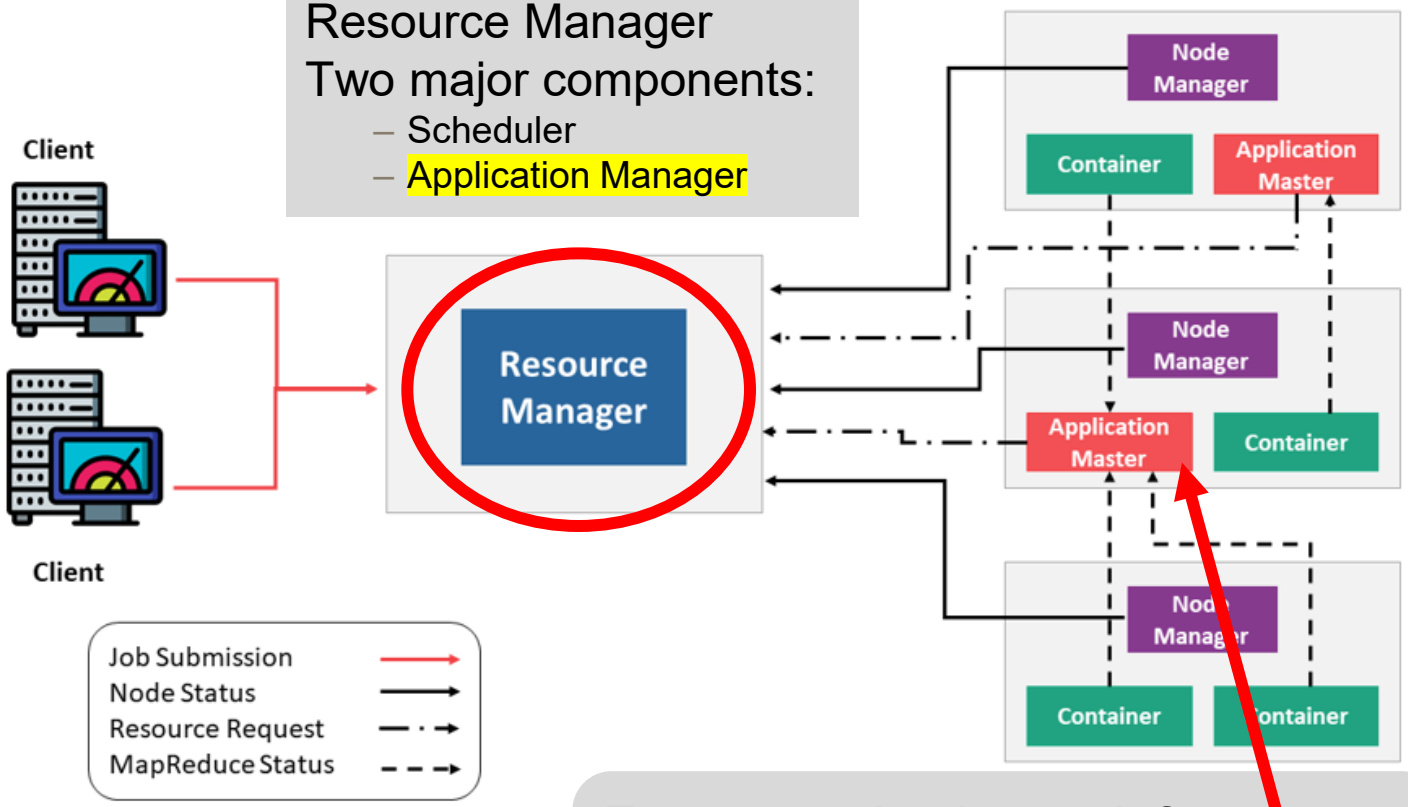


Resource Manager - Scheduler

- The scheduler is responsible for allocating resources to the various running applications based on the resource requirements
  - it does not perform any monitoring or tracking of status for the applications.
    - If there is an application failure or hardware failure, the Scheduler does not guarantee to restart the failed tasks.

Hadoop YARN

Resource Manager
Two major components:
- Scheduler
- Application Manager



For an application, ask for resource to initiate Application master

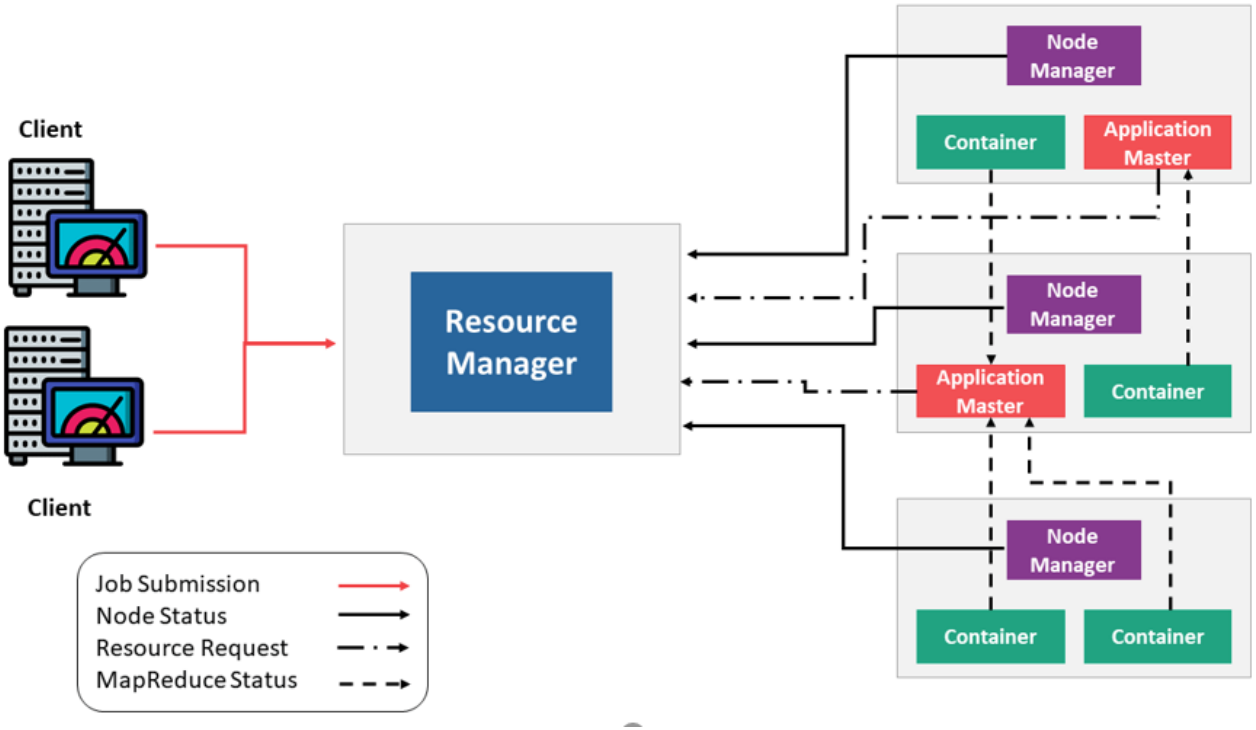
Resource Manager – Application Manager

It is responsible for accepting job submissions.

- Negotiates the first container from the Resource Manager for executing the application specific Application Master.
- Manages running the Application Masters in a cluster and
- Provides service for restarting the Application Master container on failure.

RMIT Classification: Trusted

Hadoop YARN

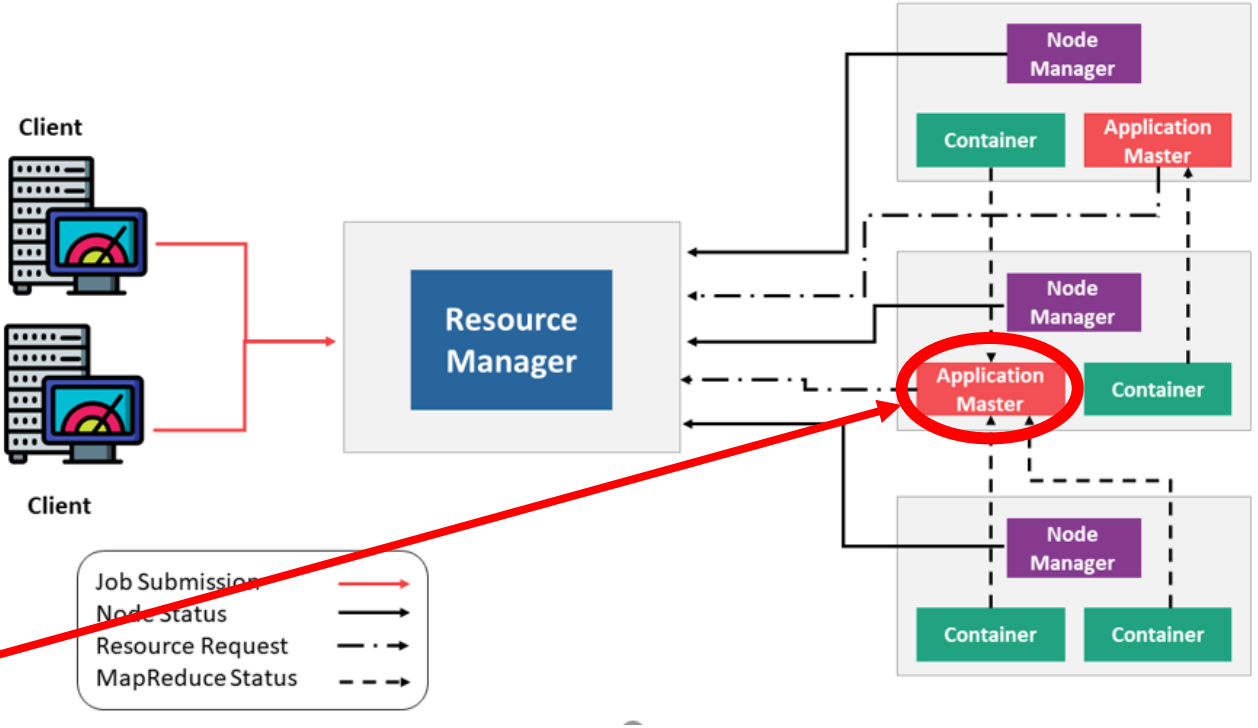


Application Master

- An application is a single job submitted to the framework.
  - Each such application has a unique Application Master associated with it

RMIT Classification: Trusted

Hadoop YARN

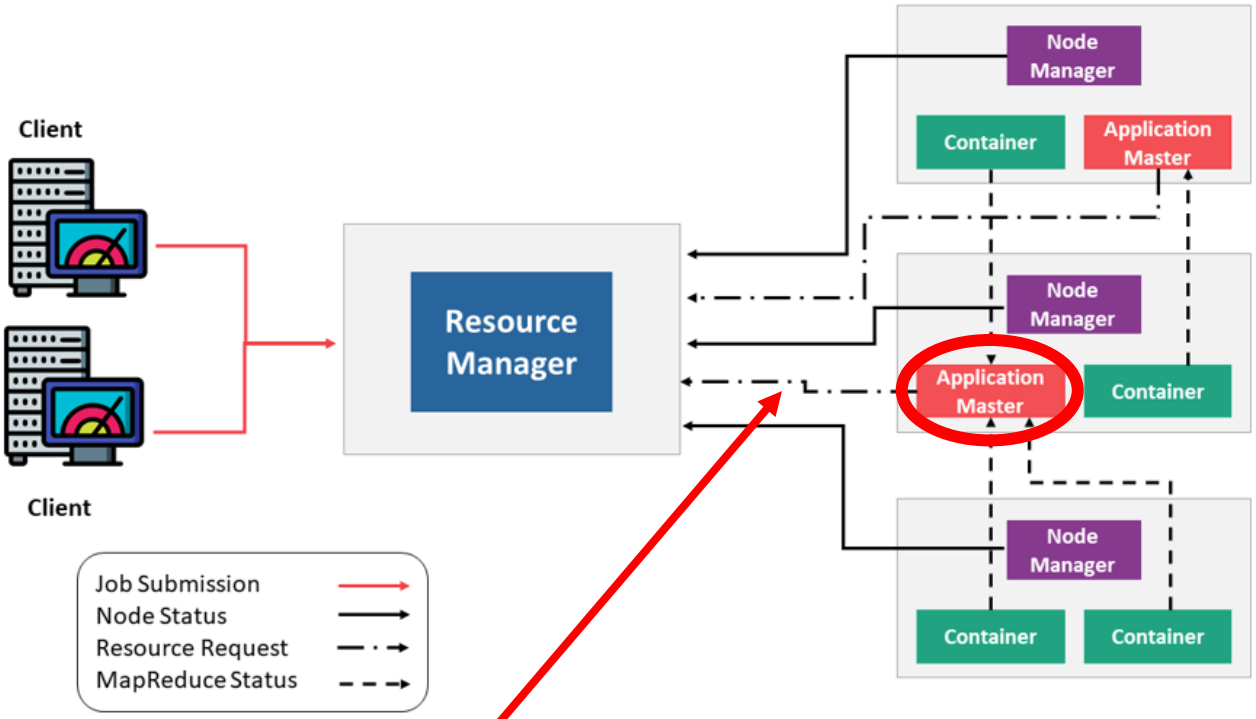


Application Master

- An application is a single job submitted to the framework.
  - Each such application has a unique Application Master associated with it

RMIT Classification: Trusted

Hadoop YARN

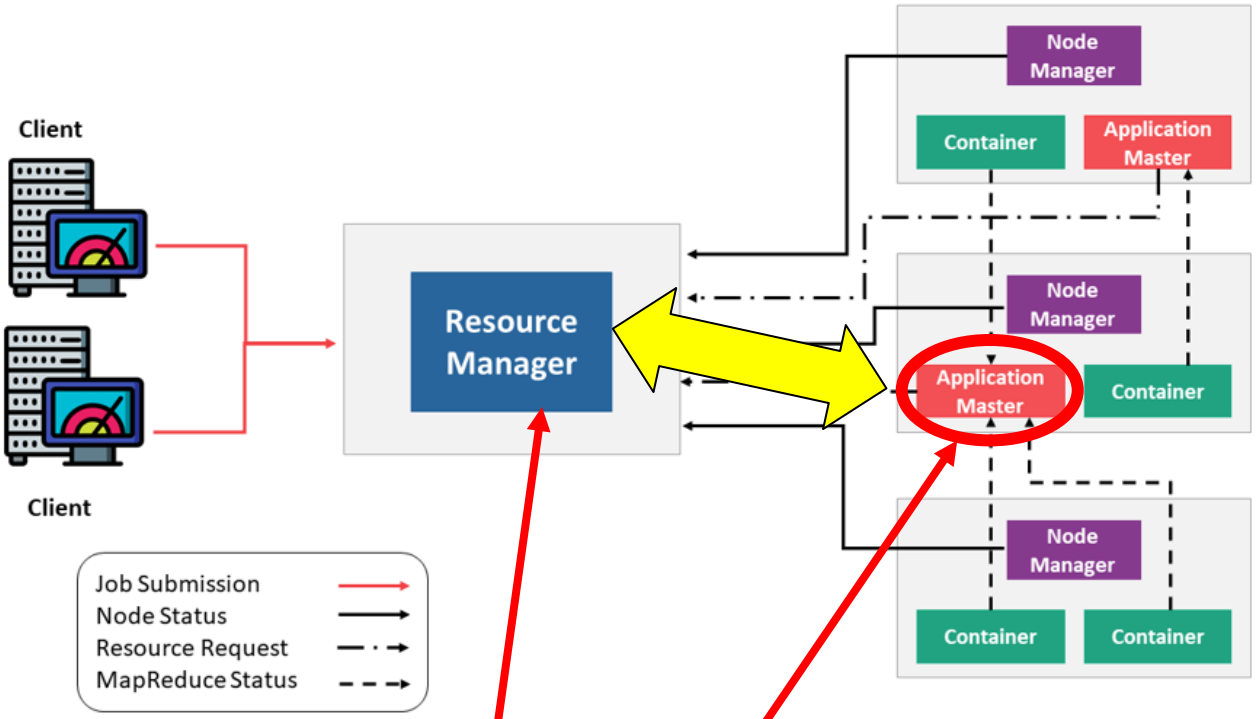


Application Master

- An application is a single job submitted to the framework.
  - Each such application has a unique Application Master associated with it
- Once started, it periodically sends heartbeats to the Resource Manager to
  - affirm its health
  - update the record of its resource demands

RMIT Classification: Trusted

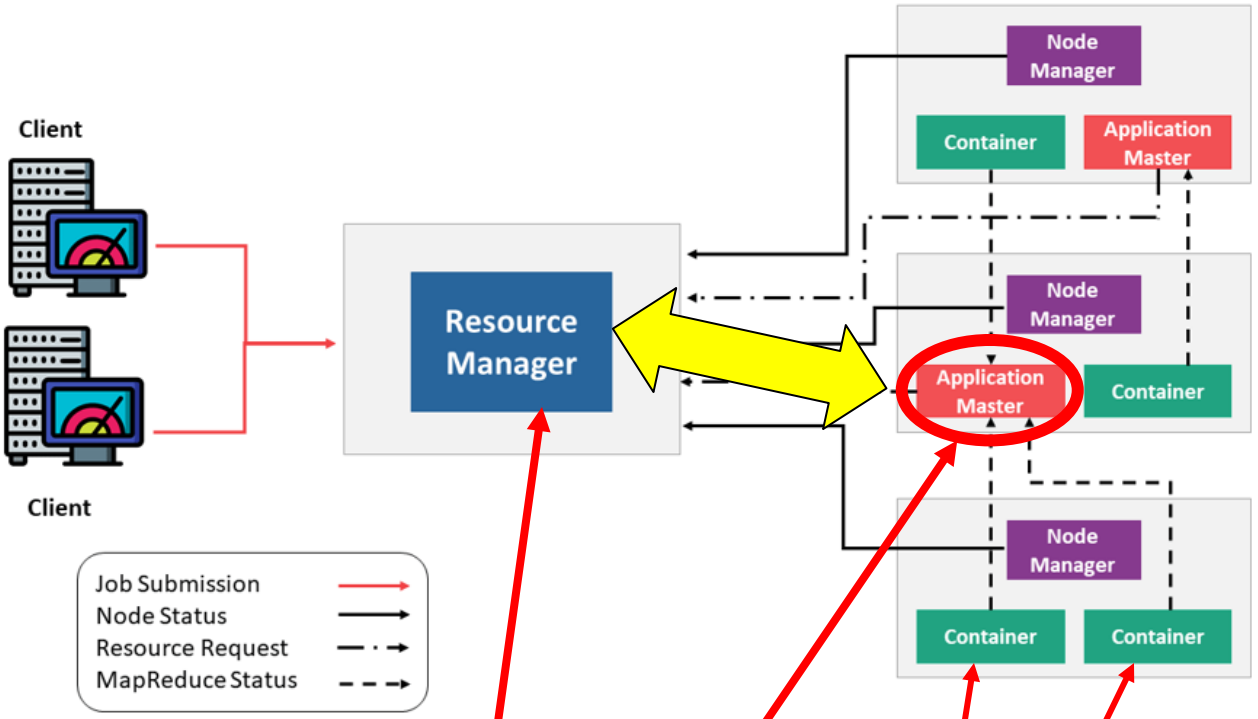
Hadoop YARN



Application Master

- An application is a single job submitted to the framework.
  - Each such application has a unique Application Master associated with it
- Once started, it periodically sends heartbeats to the Resource Manager to
  - affirm its health
  - update the record of its resource demands
    - Its task is to negotiate resources from the Resource Manager for appropriate resource containers, tracking their status and monitoring progress, and
    - work with the Node Manager to execute and monitor the component tasks.

Hadoop YARN

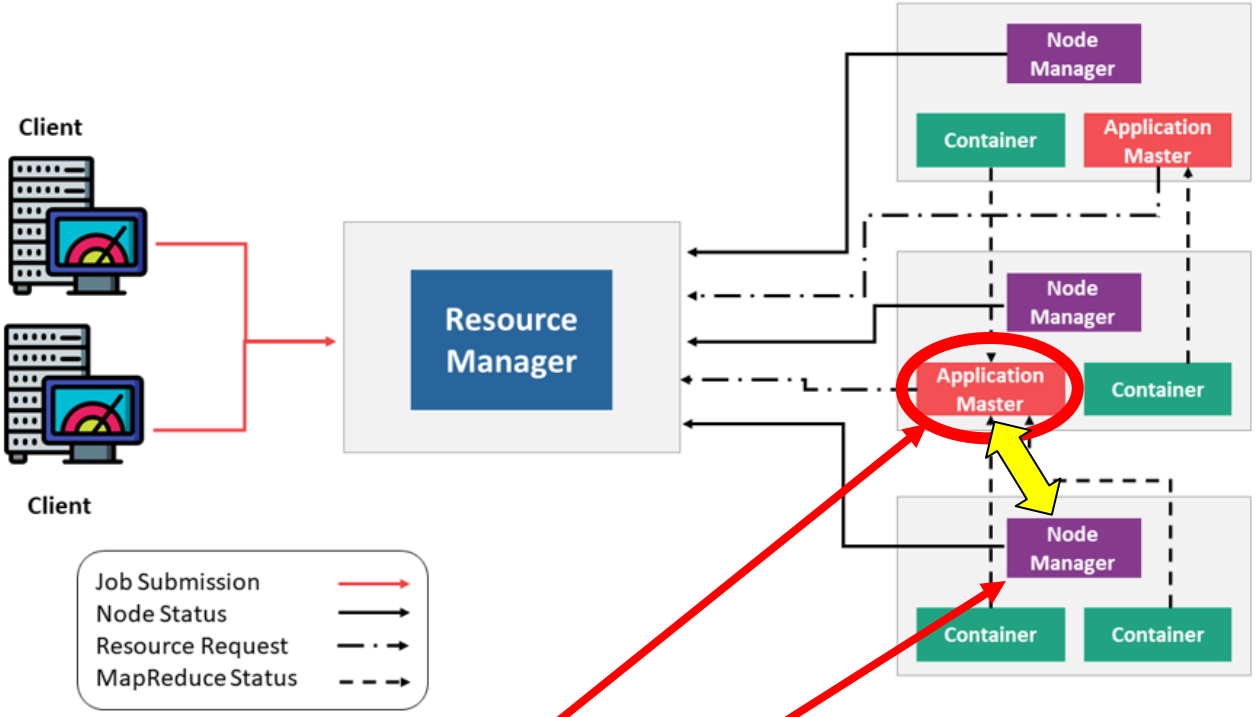


Application Master

- An application is a single job submitted to the framework.
  - Each such application has a unique Application Master associated with it
- Once started, it periodically sends heartbeats to the Resource Manager to
  - affirm its health
  - update the record of its resource demands
    - Its task is to negotiate resources from the Resource Manager for appropriate resource containers, tracking their status and monitoring progress, and
    - work with the Node Manager to execute and monitor the component tasks.

RMIT Classification: Trusted

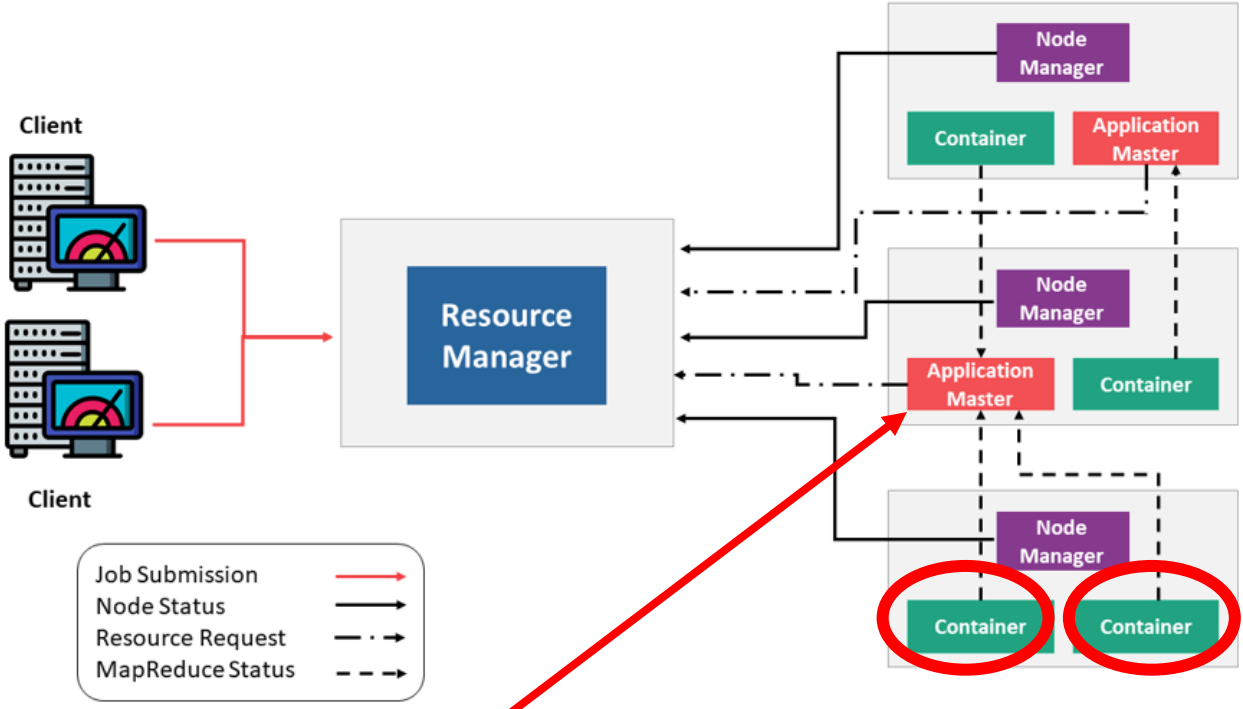
Hadoop YARN



Application Master

- An application is a single job submitted to the framework.
  - Each such application has a unique Application Master associated with it
- Once started, it periodically sends heartbeats to the Resource Manager to
  - affirm its health
  - update the record of its resource demands
    - Its task is to negotiate resources from the Resource Manager for appropriate resource containers, tracking their status and monitoring progress, and
    - Work with the Node Manager to execute and monitor the component tasks.

Hadoop YARN

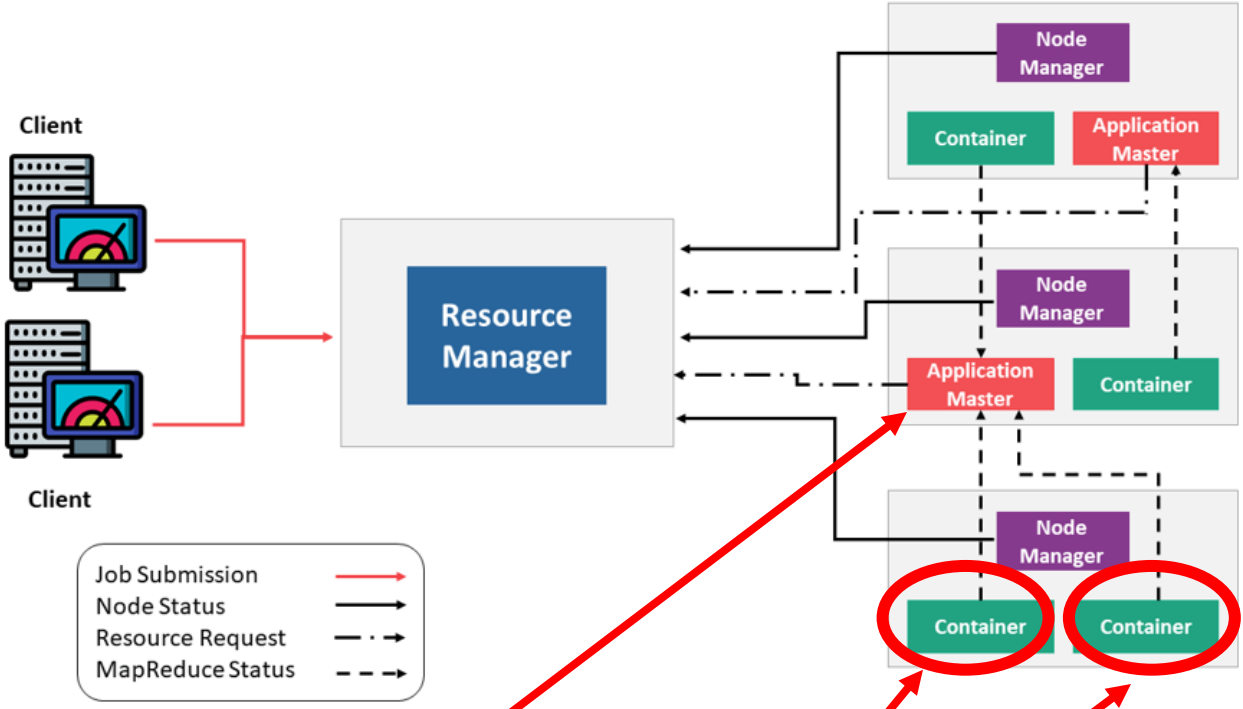


Container

It is a collection of physical resources such as RAM, CPU cores, and disks on a single node.

- YARN containers are managed by a container launch context which is container life-cycle(CLC).
- It grants rights to an application to use a specific amount of resources (memory, CPU etc.) on a specific host.

Hadoop YARN

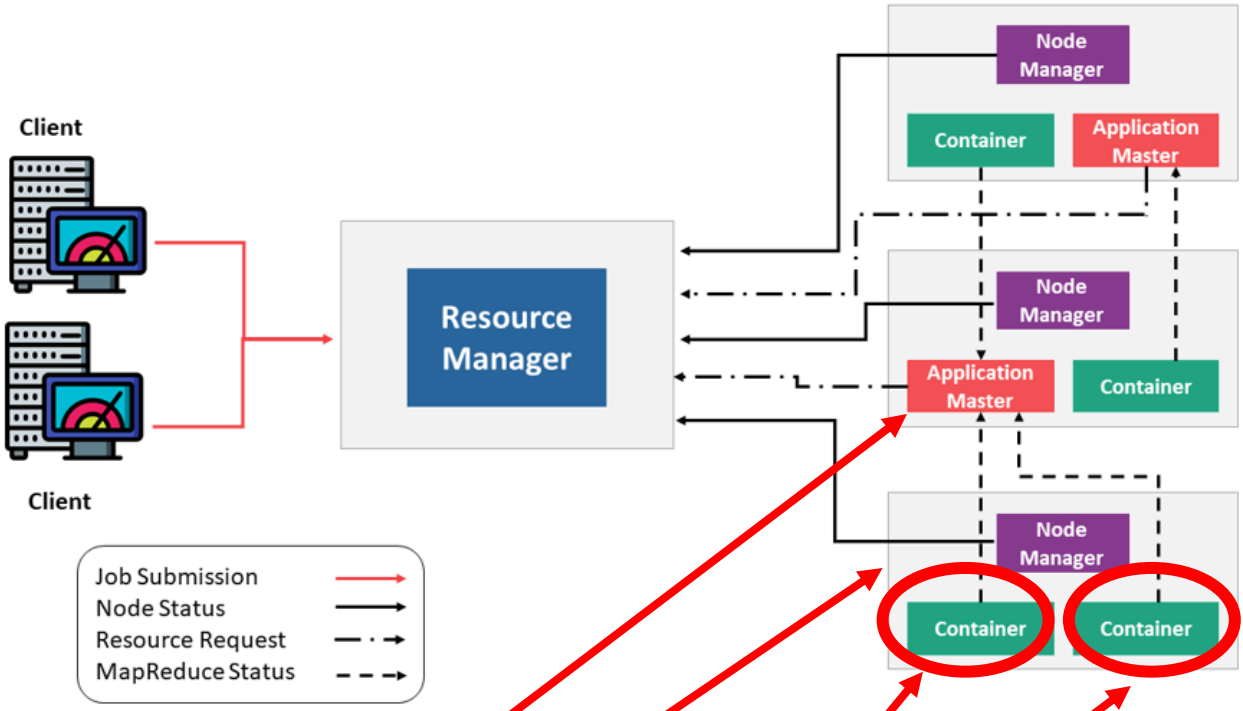


Container

It is a collection of physical resources such as RAM, CPU cores, and disks on a single node.

- YARN containers are managed by a container launch context which is container life-cycle(CLC).
- It grants rights to an application to use a specific amount of resources (memory, CPU etc.) on a specific host.

Hadoop YARN



Container

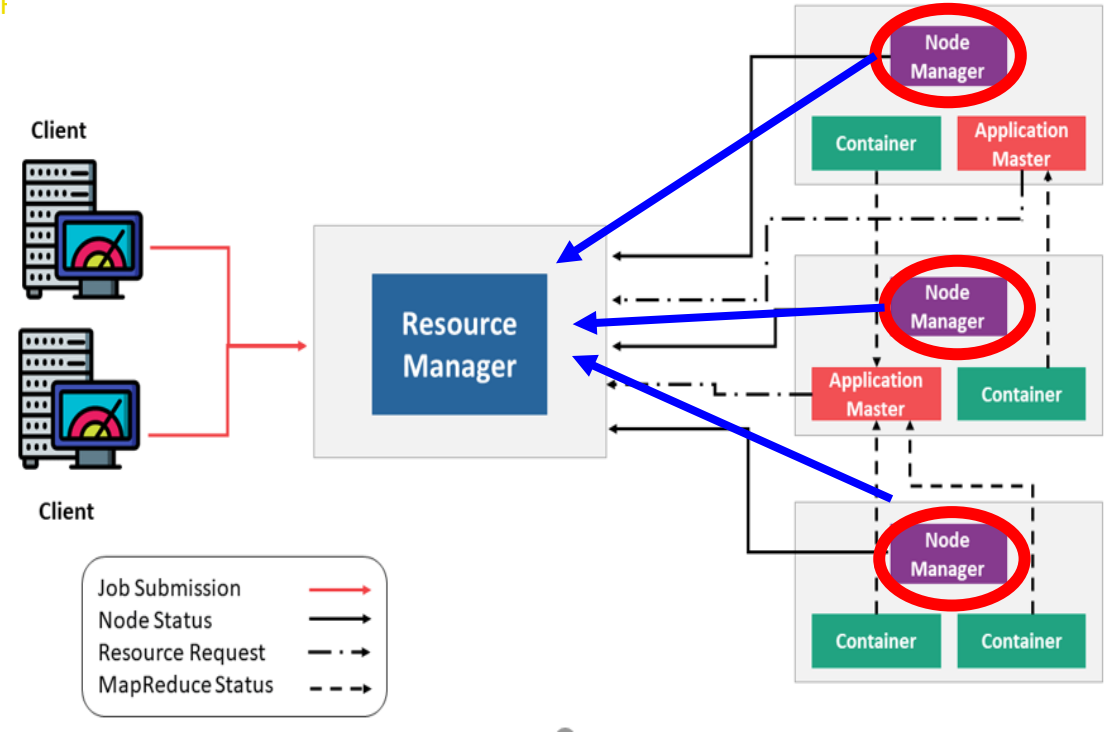
It is a collection of physical resources such as RAM, CPU cores, and disks on a single node.

- YARN containers are managed by a container launch context which is container life-cycle(CLC).
- It grants rights to an application to use a specific amount of resources (memory, CPU etc.) on a specific host.

Hadoop YARN

Node Manager

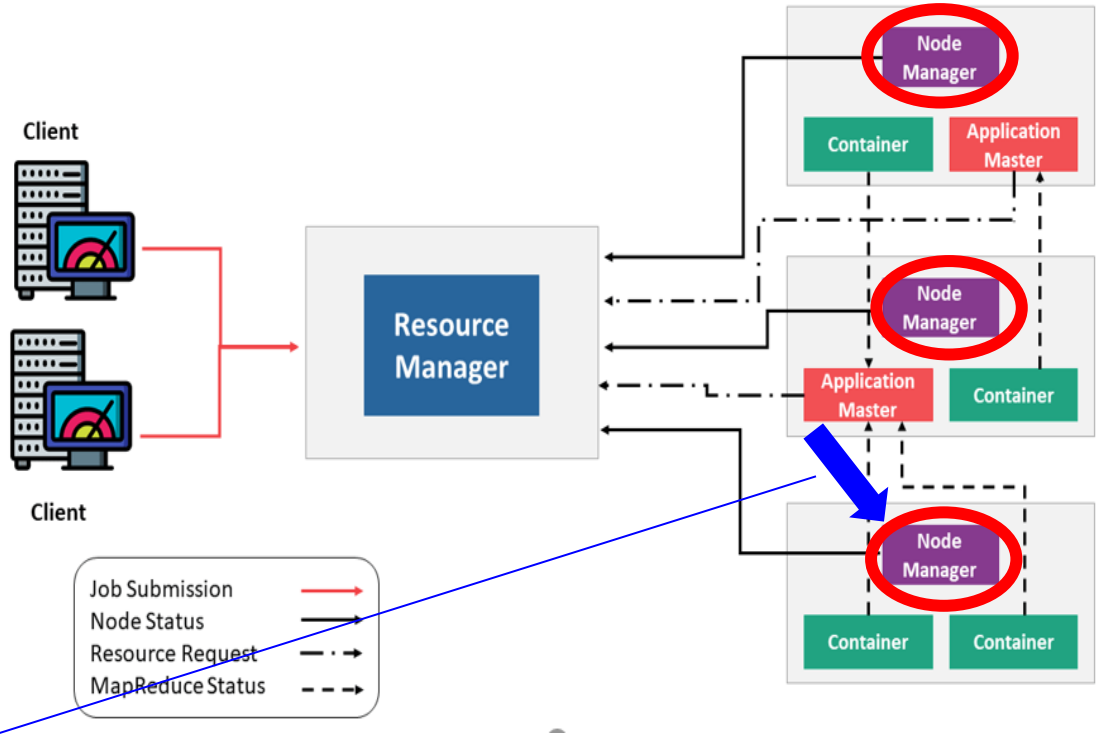
- It takes care of individual nodes in a Hadoop cluster
- It registers with the Resource Manager and sends heartbeats with the health status of the node.



Hadoop YARN

Node Manager

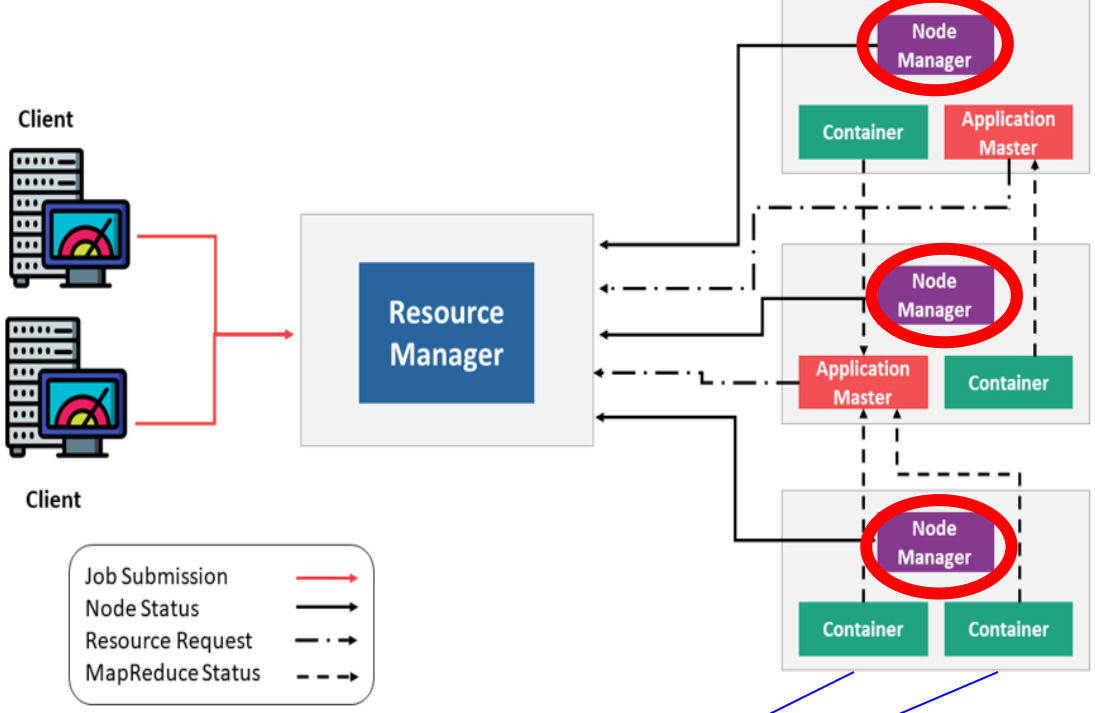
- It takes care of individual nodes in a Hadoop cluster
- It registers with the Resource Manager and sends heartbeats with the health status of the node.
- Application Master requests the assigned container from the Node Manager by sending it a Container Launch Context (CLC) which includes everything the application needs in order to run. The Node Manager creates the requested container process and starts it.



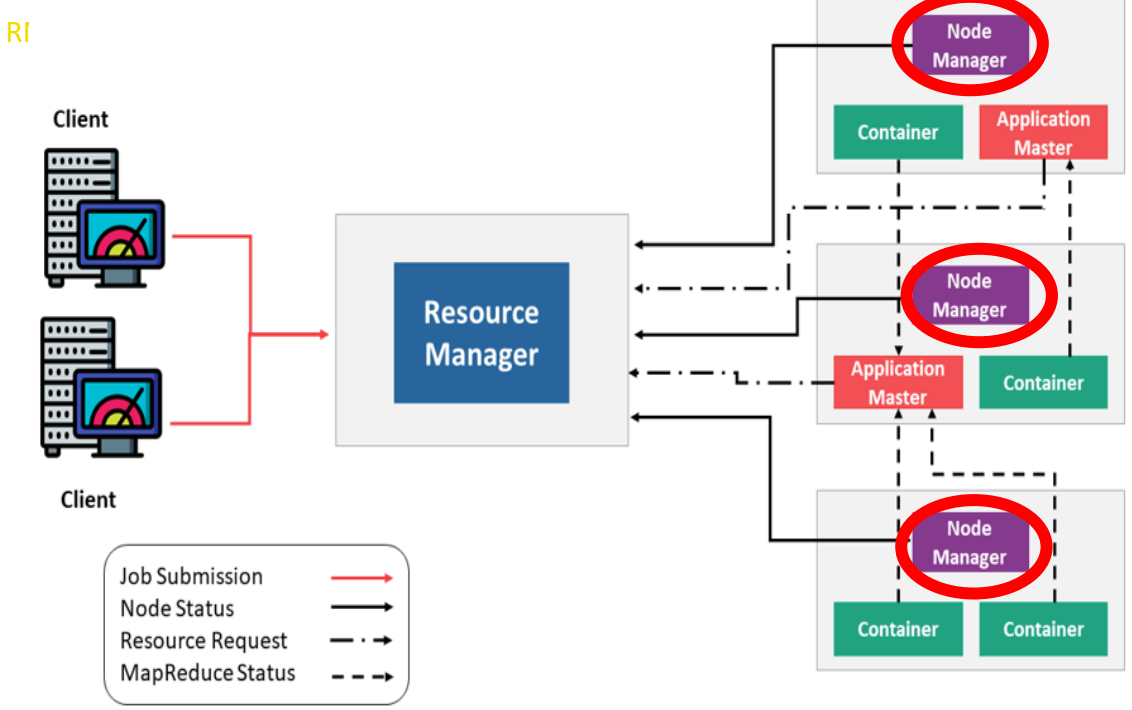
Hadoop YARN

Node Manager

- It takes care of individual nodes in a Hadoop cluster
- It registers with the Resource Manager and sends heartbeats with the health status of the node.
- Application Master requests the assigned container from the Node Manager by sending it a Container Launch Context(CLC) which includes everything the application needs in order to run. The Node Manager creates the requested container process and starts it.



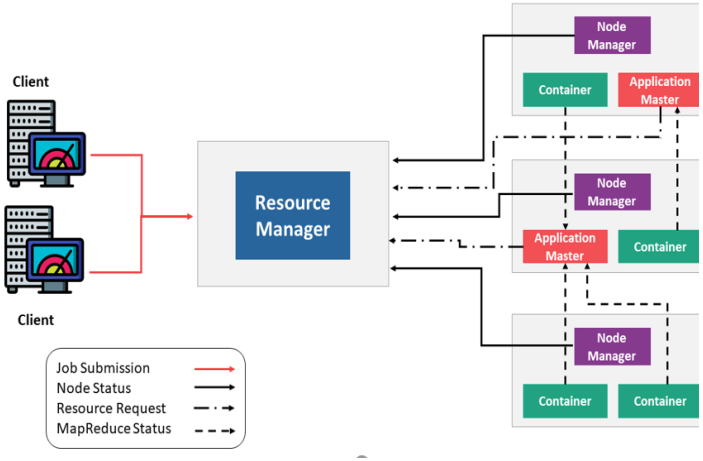
Hadoop YARN

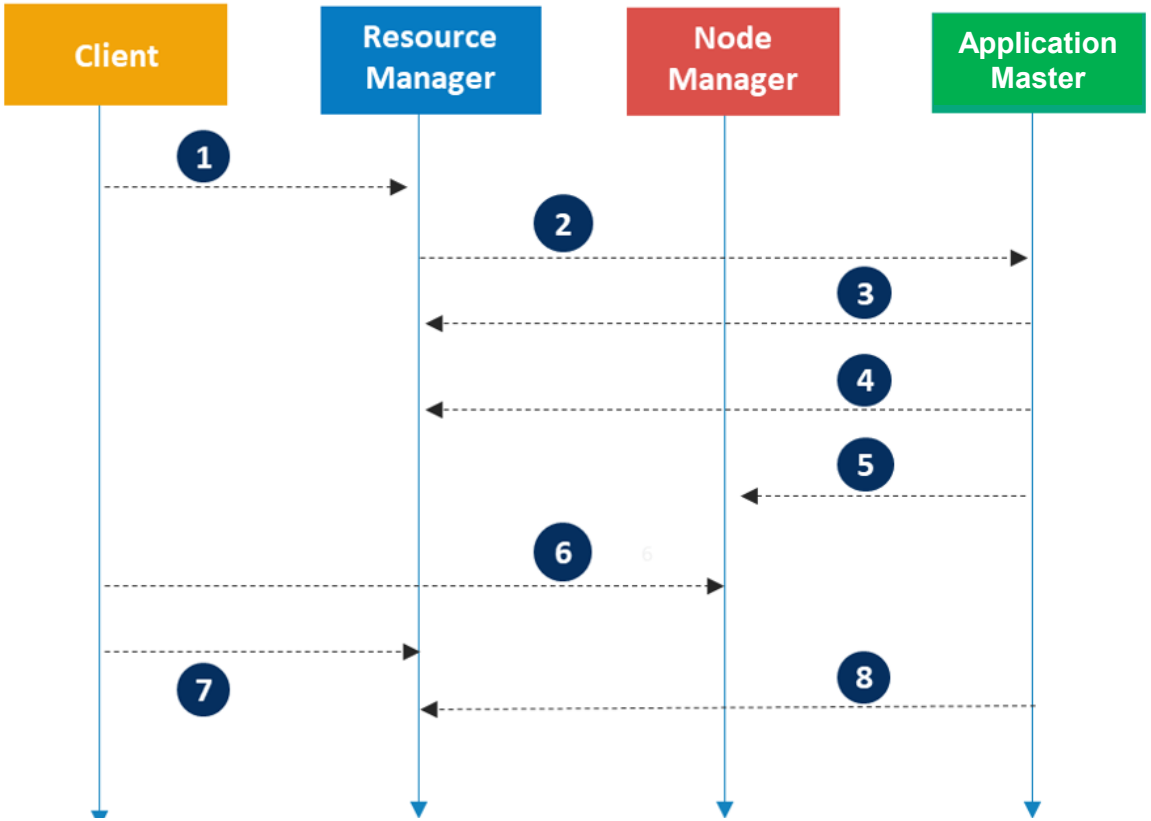


Node Manager

- It takes care of individual nodes in a Hadoop cluster
- It registers with the Resource Manager and sends heartbeats with the health status of the node.
- Application Master requests the assigned container from the Node Manager by sending it a Container Launch Context(CLC) which includes everything the application needs in order to run. The Node Manager creates the requested container process and starts it.
- Monitors resource usage (memory, CPU) of individual containers.
- It also kills the container as directed by the Resource Manager.

Hadoop YARN





- **Application Workflow**
- **in Hadoop YARN**

1. Client submits an application
2. Resource Manager allocates a container to start the Application Master
3. Application Master registers with Resource Manager
4. Application Master asks containers from Resource Manager
5. Application Master notifies Node Manager to launch containers
6. Application code is executed in the container
7. Client contacts Resource Manager/Application Manager to monitor application's status
8. Application Master unregisters with Resource Manager

End of Part 3

Next Week
MapReduce Basics